

“ALEXANDRU IOAN CUZA” UNIVERSITY OF IAȘI
FACULTY OF COMPUTER SCIENCE



BACHELOR'S THESIS

**Sequire: A Real-Time Simulation and
Visualization Platform for the BB84
Quantum Key Distribution Protocol**

submitted by

Vezeteu Andrei

Session: July, 2026

Scientific coordinator
Conf. Dr. Arusoaie
Andreea-Valentina

Avizat,

Îndrumător lucrare de licență,

Conf. Dr. Arusoaie Andreea-Valentina

Data:

Semnătura:

Declarație privind originalitatea conținutului lucrării de licență

Subsemnatul **Vezeteu Andrei** domiciliat în **România, jud. Iași, mun. Iași, strada Perju, nr. 1A**, născut la data de **18 aprilie 2004**, identificat prin CNP **5040418226834**, absolvent al Facultății de informatică, **Facultatea de informatică**, specializarea **informatică**, promoția 2026, declar pe propria răspundere cunoscând consecințele falsului în declarații în sensul art. 326 din Noul Cod Penal și dispozițiile Legii Educației Naționale nr. 1/2011 art. 143 al. 4 și 5 referitoare la plagiat, că lucrarea de licență cu titlul **Sequire: A Real-Time Simulation and Visualization Platform for the BB84 Quantum Key Distribution Protocol** elaborată sub îndrumarea doamnei **Conf. Dr. Arusoaie Andreea-Valentina**, pe care urmează să o susțin în fața comisiei este originală, îmi aparține și îmi asum conținutul său în întregime.

De asemenea, declar că sunt de acord ca lucrarea mea de licență să fie verificată prin orice modalitate legală pentru confirmarea originalității, consimțind inclusiv la introducerea conținutului ei într-o bază de date în acest scop.

Am luat la cunoștință despre faptul că este interzisă comercializarea de lucrări științifice în vederea facilitării falsificării de către cumpărător a calității de autor al unei lucrări de licență, de diplomă sau de disertație și în acest sens, declar pe proprie răspundere că lucrarea de față nu a fost copiată ci reprezintă rodul cercetării pe care am întreprins-o.

Data:

Semnătura:

Declarație de consimțământ

Prin prezenta declar că sunt de acord ca lucrarea de licență cu titlul **Sequire: A Real-Time Simulation and Visualization Platform for the BB84 Quantum Key Distribution Protocol**, codul sursă al programelor și celelalte conținuturi (grafice, multimedia, date de test, etc.) care însoțesc această lucrare să fie utilizate în cadrul Facultății de informatică.

De asemenea, sunt de acord ca Facultatea de informatică de la Universitatea "Alexandru Ioan Cuza" din Iași, să utilizeze, modifice, reproducă și să distribuie în scopuri necomerciale programele-calculator, format executabil și sursă, realizate de mine în cadrul prezentei lucrări de licență.

Absolvent **Vezeteu Andrei**

Data:

Semnătura:

Regarding the use of AI in my thesis

In line with the faculty's requirements, this section discloses how AI assistants were used during the development of this thesis. All design decisions, scientific reasoning, and the final responsibility for the content are mine; AI was used as a productivity tool under my direct review. The primary assistant was Claude (via the Claude extension), with GitHub Copilot for occasional line-level autocomplete.

Use in the codebase

AI helped with boilerplate scaffolding (REST endpoints, SQLAlchemy models, React components, FastAPI routes), debugging on specific errors I described, suggestions during larger refactors, notably the WebSocket handler refactor for alternative-protocol support and the LSTM training script, and drafting docstrings and inline comments. Every suggestion was reviewed, tested, and either accepted, modified, or rejected by me before being committed.

What was NOT delegated to the AI: the core design of the simulation pipeline, the Smart Eve feedback-control law, the LSTM architecture and training-corpus design, the "Challenge Mode" mission catalogue, and the decoy-state and finite key pipelines. All experimental results shown in chapter 8 come from simulation runs that I executed and verified myself.

Use in the documentation

The documentation was drafted, restructured, and revised in an iterative dialogue with the AI assistant, based on source material I provided (i.e. my own code, notes, and cited references). The AI also generated LaTeX scaffolding for the comparison tables, the TikZ topology diagram in section 6.1, and figure layouts. Each suggested edit was reviewed and accepted, modified, or rejected by me.

What remains entirely mine

The thesis's topic, scope, the platform's design and feature set, the choice of which QKD protocols and attacks to implement, the experimental design and reported results, the application screenshots, all source code that I reviewed and tested, and the final responsibility for every claim in this document.

Abstract

This thesis presents **Sequire**, an interactive web application, meant for studying the distribution of quantum keys (also known as QKD), a method of exchanging cryptographic keys, whose security comes from the laws of physics, rather than the mathematical hardness of some math problems. The platform implements the **BB84 protocol**, alongside three other QKD-related protocols, allowing the user to execute any of these either as a software simulation, or directly on an IBM quantum processor. Every execution is unfolding bit by bit in browser, accompanied by a security analysis which decides whether the resulted key is secure or not. A separate section named **Challenge Mode** completes the main interface. Within it, the user must identify an attack starting from the behaviour of the protocol, or to configure the system themselves in order to hit a special target. The scope of the project is to make accessible, through an ordinary web browser, a subject that usually requires either a specialized hardware, or a research software.

Motivation

When I began to look into what comes next for cryptography, the answer was always the same: quantum computers will eventually break the schemes we rely on today. The natural move would be shifting to post-quantum algorithms, which are also based on hard mathematical problems, but which are believed to withstand quantum attacks. **Quantum Key Distribution** (QKD) is a more uncommon answer: it does not patch classical cryptography, but replaces the hypothesis it is built on. Instead of trusting that an adversary cannot solve a hard problem, QKD trusts the laws of physics. That shift in foundation is what made me want to study QKD.

The gap that I wanted to close with my thesis is not theory itself, but the accessibility. Real QKD hardware is well out of reach for most universities, and the software simulators are either built for researchers or are too simplified to teach something substantial. There was room for an interactive platform, browser-accessible, that walks the user through the entire BB84 pipeline, end-to-end, which can also send the same circuits to the IBM Quantum Hardware when solicited. **Secure** is the answer to this gap.

My own experience with QKD started from textbooks and academic papers, and what stayed with me was rarely a definition. It was the moment a concept clicked because I saw what changed when I tweaked a parameter. I wanted a tool where a student can move a slider, watch the bit error rate climb or plummet, see when a channel goes from secure to compromised, and understand why, without first setting up a python environment or reading hundreds of pages of preliminaries. That is why I decided to build this platform.

What follows are the steps of how that platform was built: the protocols it implements, the design choices behind the simulation engine, the post-processing layer that turns noisy measurements into a usable key and the eavesdropper-detection mechanisms that put the security claims to the test. The chapters that follow explain why each piece had to be where it is.

Contents

1	Introduction	9
2	Application Context and Problem Statement	13
2.1	The Problem	13
2.2	What Sequire Does	14
2.3	Goals	14
2.4	The Three Actors	15
3	Theoretical Background	16
3.1	Post-Quantum Cryptography vs QKD	16
3.2	Quantum Computing Foundations	16
3.2.1	Qubits and Quantum States	16
3.2.2	Measurement and State Collapse	17
3.2.3	Hadamard Gate and the Diagonal Basis	17
3.2.4	Pauli Gates	18
3.3	The No-Cloning Theorem	18
3.4	Heisenberg's Uncertainty Principle in QKD	18
3.5	Quantum Entanglement	19
3.6	Quantum Bit Error Rate (QBER)	19
3.7	Error Correction & Privacy Amplification	20
3.8	The BB84 Protocol	20
3.8.1	Error Correction	21
3.8.2	Privacy Amplification	21
3.9	The CASCADE Information-Reconciliation Algorithm	22
3.10	Optical Fibre Channel Attenuation	22
3.11	Decoy-State Method & Photon Number Splitting Attack	23
3.12	Machine Learning for QKD Security Monitoring	24
3.13	Alternative QKD Protocols	25
3.13.1	B92 (Bennett, 1992)	25
3.13.2	SARG04 (Scarani-Acín-Ribordy-Gisin, 2004)	25
3.13.3	E91 (Ekert, 1991) and the CHSH Bell Test	26
3.14	Finite Key Security Bounds	27
4	Existing QKD Simulators and Related Work	28
4.1	NetSquid	28
4.2	QuNetSim and SimulaQron	28
4.3	Qiskit Textbook Demos	28
4.4	Positioning of Sequire	28

5	Technologies Used	30
5.1	Python 3.11	30
5.2	Qiskit & Qiskit-Aer	30
5.3	Qiskit IBM Runtime	30
5.4	FastAPI	31
5.5	WebSockets for Real-Time Streaming	31
5.6	SQLite and SQLAlchemy	31
5.7	PyTorch	31
5.8	React 19 and Vite	32
5.9	Tailwind CSS	32
5.10	Recharts	32
5.11	Authentication Stack	32
6	Application Architecture	34
6.1	High-Level System Overview	34
6.2	Backend Component Structure	34
6.3	Frontend Component Structure	35
6.4	Database Schema	35
6.5	Request and Event Flow	36
7	Implementation and Features	38
7.1	Configuration Panel	38
7.2	The BB84 Protocol Engine	41
7.3	Eve Eavesdropping Simulation	41
7.4	Real-Time WebSocket Streaming and PhotonGrid	42
7.5	Key Distillation Pipeline	42
7.6	Decoy-State and PNS Attack Panels	42
7.7	Finite Key Security Panel	43
7.8	Machine Learning Detection Panels	43
7.9	Bell Test Panel (E91)	44
7.10	QBER Historical Chart and Key Rate Curve	44
7.11	Educational Panel and Fun Facts	44
7.12	Challenge Mode	45
	7.12.1 Mission Catalogue	45
	7.12.2 Grading	45
	7.12.3 Progress and Leaderboard	46
7.13	Session Metadata Widget	47
8	Experimental Results	48
8.1	Methodology	48

8.2	QBER under Different Eve Modes	48
8.3	Impact of Noise Parameters on Key Yield	49
8.4	Key Rate vs. Channel Distance	49
8.5	Hardware Validation on IBM Quantum Hardware	50
8.6	B92 Sifting Efficiency	51
8.7	E91 CHSH Parameter	51
8.8	LSTM Detector Performance	51
9	Conclusions	52
9.1	General Achievements	52
9.2	Future Development Possibilities	53
9.3	Closing Remarks	53

1 Introduction

Cryptography is the practice of developing and using coded algorithms to protect and obscure transmitted information so that it may only be read by those with the permission and ability to decrypt it. In practice, cryptography is the "invisible shield" of the digital world. It transforms readable data into a long string of characters (ciphered text).

If we put aside the math behind it, cryptography fulfills four essential functions:

- **Confidentiality:** It ensures that nobody besides the two communicating parties can read any message;
- **Integrity:** Guarantess that the data was neither modified nor truncated while it was transferred;
- **Authentication:** It proves that the site or entity you want to communicate with is actually who they pretend to be;
- **Non-repudation:** It ensures that someone who sent a message or has signed a document cannot deny this fact afterwards.

Cryptographic algorithms are mathematical procedures and sets of logical rules that are used for securing information by transforming readable data into secret codes, guaranteeing their confidentiality, integrity and authenticity. Cryptographic algorithms are classified in three categories. Firstly, there are symmetric algorithms, who use a single secret key to quickly encrypt and decrypt large amounts of data, where both the sender and the receiver use the same secret key to "hide" the data, but also to reveal the data; some well-known symmetric algorithms are AES, ChaCha20 and 3DES. Then there are the asymmetric algorithms, the ones using a pair of keys (public and private) in order to securely exchange secret keys and to apply digital signatures; some widely-used asymmetric algorithms are RSA, ECC and Diffie-Hellman. Lastly, there are hash functions, that transform any amount of data into a fixed-length string (a one-way, unique and digital signature) , used only to verify the integrity of data and that of passwords; some popular hashing functions are SHA-256 (section 3.7), SHA-3 and MD5/SHA-1. A few cryptographic algorithms to keep in mind are:

- **AES** (Advanced Encryption Standard) [1]: It is the most widely used symmetric algorithm in the world, famous for its speed and extreme efficiency, being the global standard, used by various governments, banks and applications (e.g. Whatsapp) to secure confidential data;
- **RSA** (Rivest-Shamir-Adleman) [2]: This is the pioneer of asymmetric cryptography (with both public and private keys), based on the mathematical difficulty of factorizing big prime numbers, being the algorithm allowed the securization of

the first transactions and internet connections;

- **ECC** (Elliptic Curve Cryptography) [3]: It's an asymmetric algorithm that uses the mathematical properties elliptic curves in order to offer the same level of security as RSA, but with way smaller keys, which makes it ideal for smartphones and limited-resource devices.

Classic cryptography is based on a security that is purely mathematical and also temporary, as it protects data through complex mathematical problems that current technology finds difficult to solve, leaving systems vulnerable to new algorithms or the inevitable rise of quantum computers, which will be able to instantly crack foundational standards like RSA or ECC using Shor's algorithm. Additionally, it suffers from the challenge of securely distributing secret keys and the absolute impossibility of detecting interceptions in real time, which allows hackers to passively copy and store encrypted data today to decrypt it later, without the sender or receiver ever finding out.

The transition from classical cryptography to **quantum cryptography** shifts the digital security from the mathematical barriers to the laws of physics. While the classical systems are based on complex equations which quantum computers will eventually crack sometime in the future, quantum cryptography is the science that deals with securing the data through by encoding the information into quantum states of the light particles (photons). This technology uses properties like superposition and quantum inseparability, guaranteeing that the state of said particles is going to modify at any intercepting attempt.

In practice, this phenomenon offers a very high security and introduces an unique "alarm system": any cyber attack will leave a fingerprint that is instantly detectable. Therefore, information security does not "depend" on the computing power of an attacker, but becomes a certainty, assured by nature's law, immune in front of any quantum computer.

A **protocol** is a set of rules which two or more parties have to follow, in order to accomplish a common task. In the context of cryptography, it describes exactly what each party sends, what do each of them receive and what do they do with the information, such that in the end they'll obtain the desired result (e.g. a common key)

Sequire implements four QKD protocols representing two complementary paradigms: *prepare and measure*: the **BB84** protocol and its variants, **B92** and **SARG04**, as well as *entanglement-based*, the most sensible choice being the **E91** protocol. This dual coverage lets the user compare not just the protocol variants, but two fundamentally different routes to quantum key distribution. Each of these protocols illustrates a different aspect of quantum cryptography and can be put head to head within the same platform. **BB84** is the reference-protocol, created by **Charles Bennett** (physicist) and **Gilles Brassard** (computer scientist) in 1984 [7], being the first historically one to ever

be proposed and also the one that this entire application "revolves" around. The other three are alternatives either extend or modify the original approach. I included them because they allow direct comparisons within the same platform: same experimental conditions, same attacks, same interface, but different internal mechanisms. Thus, my thesis is not limited to only one implementation, but offers a "bird's-eye view" upon the many directions which the quantum cryptography has taken over the past forty years.

The idea behind **BB84** is simple. Two parties who want to exchange a secret key use quantum particles (photons) as information carriers. One of them encodes each bit of the key into a quantum state of a photon, choosing from several possible bases. The other entity measures every received photon, randomly choosing the base. In the end, both of them publicly announce their bases and only the bits for whom they have chosen **the same basis**. These bits make up the common key. The protocol's security is not based off a mathematical hypothesis, but on a physics principle. Any attempt of a third party to read the photons through transport will modify their quantum state, leaving a measurable "mark" in the resulting key.

However, no matter how good it is implemented, a QKD protocol will **never** produce an absolute perfect key. A portion of those bits may be wrong, either due to the natural noise of the channel or because of an adversarial intervention. To evaluate this, the two parties a small subset of bits from the obtained common key and check how many bits match. The percentage of the bits that do not match is called **QBER** (short for Quantum Base Error Rate). QBER is the main security measure in QKD. If its value is below a certain threshold, usually set to 11% [8], then the two entities can conclude that the transmission was secure enough so that they can use the key further; but if it exceeds the threshold, the trnasmission is abandoned and the protocol begins all over again.

Throughout the whole thesis I will be using the standard names from cryptography literature. **Alice** and **Bob** are the two legitimate entities that want to exchange a secret key. **Eve** is the third actor, the attacker who wants to intercept the key on its way, without Alice and Bob noticing. Alice has the role of the sender. She generates the bits randomly and encodes them into quantum states, which are further sent on the quantum channel. Bob has the role of the receiver. He measures the received states and takes part at the necessary processing steps in order to obtain the final key. Eve is able to make any operation that is allowed by the laws of physics: measure the photons in the channel, can replace them with other bits, can store them in the quantum memory or can copy parts of them, when the source allows it.

Also, we can't have the full picture without understanding how the attacks work. As far as my thesis is concerned, two classes of attacks appear. The **intercept-resend**

attack is the canonical active attack on BB84: Eve measures every qubit in a base of her choice and forward a fresh one to Bob in the state corresponding to her outcome. Since Eve picks the wrong base half the time, this attack inevitably introduces detectable errors into Bob's key. A full intercept-resend raises the QBER to around 25%, which is well above any realistic noise limit. **Sequire** parameterizes it as **Weak**, **Strong** and **Smart** Eve, the last one actively keeping the QBER below the detection threshold. The **Photon Number Splitting** (PNS) attack is more subtle and specific to imperfect physical sources: real implementation use attenuated lasers that occasionally emit multi-photon pulses. Eve counts the photon in every pulse, blocks the single-photon ones and keeps a copy of each multi-photon pulse for later measurement, once Alice and Bob announce their bases. PNS leaves the QBER unscathed and would compromise long-distance BB84 entirely if not for the decoy-state defence [15, 16] (see section 3.11).

A second principle of quantum mechanics that QKD can use is **quantum entanglement**: two particles can be prepared in a common state such that measuring one instantly fixes the outcome of the other, no matter how far apart they are. This correlation is stronger than anything classical pre-arrangement can ever produce. Alice measures one photon and Bob the other, and the perfect correlation of their outcomes both yields the shared bits and certifies that no eavesdropper has disturbed the pair.

Sequire brings these elements together in a single browser-based platform. The user is able to choose and then configure a QKD protocol from a sidebar with seven preset scenarios, such as lab bench, metro link, satellite uplink, adversarial, as well as one preset per alternative protocol, and a set of free parameters that include noise sliders, channel distance, photon source type, error-correction method, and the adversary model (section 7.1). Each run unfolds cell by cell on a live grid, streamed over a WebSocket connection (section 7.4), and the user observes the key being distilled step by step from raw qubits via sifting, error correction, and privacy amplification (section 3.7) to a final 128-bit secret key. Beyond the four protocols mentioned above, the platform supports execution either on the Qiskit-Aer noise simulator (section 5.2) or directly on real IBM Quantum superconducting processors via Qiskit IBM Runtime (section 5.3). It implements five adversary models: None, Weak, Strong, Smart, and PNS (introduced in section 2.4, implemented in section 7.3), three layers of analytical depth: optical fibre attenuation (section 3.10), decoy-state bounds (section 3.11), finite-key bounds (section 3.14), two machine-learning eavesdropping detectors (Random Forest and LSTM, section 3.12), a Bell test panel for E91 (CHSH inequality check, section 7.9), and a gamified Challenge Mode with fifteen procedurally generated missions (section 7.12), meant to test the user's knowledge. Authenticated users persist their simulation history and Challenge Mode progress in a local SQLite database (section 6.4), with sign in through email w/ password or Google OAuth (section 5.11).

This thesis is structured as follows. **Chapter 2** states the problem that **Sequire** addresses, sketches what the platform does, what goals it has and introduces the three cryptographic actors (Alice, Bob, Eve) referred to throughout the thesis. **Chapter 3** develops the theoretical background of QKD, from quantum-computing foundations to BB84, error correction and privacy amplification, channel attenuation, the decoy-state defence, finite-key security bounds, machine-learning detection, as well as the three alternative protocols. **Chapter 4** surveys existing QKD simulators (NetSquid, QuNetSim, SimulaQron, and the Qiskit notebooks) and positions **Sequire** relative to the state of the art. **Chapter 5** documents the technology stack on which the platform is built. **Chapter 6** describes the three-tier application architecture, the backend and frontend module organisation, the database schema, and the end-to-end request flow. **Chapter 7** walks through every implemented feature panel by panel, from the configuration controls and the BB84 engine to the Bell test panel and the Challenge Mode. **Chapter 8** reports the experimental measurements, including QBER under each adversary model, the impact of noise on key yield, key-rate versus distance, the CHSH parameter for E91, the LSTM detector performance, and a hardware validation run on real IBM Quantum processors. **Chapter 9** summarises the contributions of this work and outlines directions for future development.

2 Application Context and Problem Statement

2.1 The Problem

Quantum key distribution has been studied for decades (BB84 has been around since 1984, so definitely has been a long time since) but it remains hard to learn for most students, and there are two main reasons why. Firstly, real QKD hardware is very expensive and only available in a few research labs; a student at an ordinary university has no realistic way to see the protocols running on actual qubits. Second, the software tools that simulate QKD fall into two extremes. On one side, frameworks like **NetSquid** and **QuNetSim** are written for researchers and require setting up a Python environment, reading a multitude of pages of documentation and understanding the simulator's internal model before any output can be produced. On the other side, the Qiskit textbook notebooks show a short BB84 example in a few cells, but it stops short of the full protocol: there is no step that corrects the inevitable errors in the shared bits, no step that shrinks the key so an attacker who listened in on the channel cannot reconstruct it, no defence against advanced attacks on a real laser source, no eavesdropper detection beyond a basic error-rate comparison, and no per-qubit visualisation of the exchange.

The result is that a student who wants to understand BB84 from one end to another has to either accept the protocol on faith from a textbook or spend weeks in a tool meant

for researchers. Neither path teaches the protocol in the way most people actually learn: by moving a slider, watching what changes, and asking why.

2.2 What Sequire Does

Sequire is a web application that walks the user through the full BB84 pipeline, qubit by qubit, in any modern browser. The user opens the page, picks a protocol from a sidebar, sets up a few parameters (the qubit count, the noise level, the channel distance, the presence of an eavesdropper), and presses one button. The page then shows each qubit being prepared by Alice, sent across the public channel, and measured by Bob. The error rate on Bob's side, the bits Alice and Bob both kept after comparing their basis choices, the bits that remain after errors are corrected, and the final shortened secret key are reported as they are produced.

In addition to BB84, **Sequire** also implements three other QKD protocols: B92, SARG04, and E91. The user can run any of these on either a software simulator (Qiskit-Aer with a configurable noise model) or, when IBM Quantum credentials are provided, on a real superconducting quantum processor (through the Qiskit IBM Runtime). Same interface, same parameters, and the same reported metrics are used in both modes.

2.3 Goals

Sequire is built first as a teaching tool. A student should be able to use it to:

- watch how a BB84 run produces a shared key, step by step;
- see how the error rate on Bob's side reacts to noise, channel distance, and eavesdropping;
- compare the four protocols (BB84, B92, SARG04, E91) on the same configuration and see what changes;
- observe how attacks (Eve measuring each qubit and sending Bob a fresh fake copy, or Eve taking advantage of a real laser source) affect the channel verdict;
- watch the defence based on sending pulses at two different intensities catch attacks that a simple error-rate comparison alone would miss;
- see how machine-learning detectors flag attackers that stay below the error-rate cut-off;
- run the same protocol on real IBM hardware and compare the result with the simulator.

A second goal is to provide a working bench that a developer or researcher can use to test ideas quickly: change a parameter, run a batch of simulations, look at the error-rate history or the feature-importance scores of the machine-learning detector.

The Challenge Mode, described later in this thesis, builds on this bench by turning it into a small game of configurable missions.

2.4 The Three Actors

Throughout the rest of this thesis the protocols are described using the three standard cryptographic characters.

Alice is the sender. She holds the qubit source, generates a random sequence of bits, encodes each bit into a qubit (in BB84, by picking one of two possible measurement bases at random), and sends the qubits one by one across the quantum channel. In **Sequire**, Alice's random choices are drawn from a pseudo-random number generator that the user can seed for reproducibility.

Bob is the receiver. He measures each incoming qubit in a basis of his own random choice, records the outcome, and later exchanges his basis choices with Alice over a classical public channel. The bits where Alice and Bob picked the same basis form the shared raw key; the rest are discarded. In **Sequire**, Bob's measurement is executed on the Qiskit-Aer simulator or on the IBM hardware, depending on the mode the user has selected.

Eve is the eavesdropper. She tries to learn the key by acting on the quantum channel: she can measure each intercepted qubit and send Bob a fresh, fake copy in its place, or, when **Alice** uses a real laser source instead of an ideal single-photon emitter, she can take advantage of the rare pulses that contain more than one photon (she keeps one photon for herself and lets the others through to Bob). The laws of quantum mechanics make it impossible for Eve to do this without leaving a trace; the size of that trace is what the rest of the thesis measures. **Sequire** exposes five Eve modes: None (Eve is absent), Weak (Eve intercepts and resends only a fraction of the qubits), Strong (Eve intercepts and resends every qubit), Smart (Eve adapts her behaviour to keep the visible error rate just below the level at which the protocol would abort), and PNS (Eve targets the real laser source by skimming photons from multi-photon pulses).

3 Theoretical Background

3.1 Post-Quantum Cryptography vs QKD

In 1994, **Peter Shor** proved that a quantum computer that is large enough could factorize very large integers and solve the **discrete logarithm problem** [5], breaking RSA and ECC. Existing quantum hardware is not yet large enough, but the gap is getting smaller and smaller. Encrypted traffic captured today can be stored and decrypted retroactively once the threshold is reached. This is the **"harvest now, decrypt later"** concern (i.e. "capture encrypted traffic now, and decrypt it later, once quantum computers exist").

Post-quantum cryptography (PQC) stays in software: it replaces RSA and ECC with new schemes based on harder math problems, standardised by NIST (National Institution of Standards and Technology) in 2024. Its security remains computational, with no proof that the quantum algorithms cannot break the new problems either.

Quantum Key Distribution (QKD) replaces the hypothesis itself. Its security is informational-theoretic and is derived from physics, not from math, but it requires a quantum channel and dedicated hardware. QKD does not replace classical cryptography, instead it distributes the symmetric keys that algorithms like AES then use to encrypt the actual traffic. This chapter develops the QKD route, which is the one **Sequire** studies.

What makes both Shor's attack and the protection offered by QKD is the same shift in the below-layer. Information is no longer carried by classic bits, but by **qubits**, which obey other rules, rules that classical computation simply does not have. The next subsection sets out the necessary **quantum computing** vocabulary, which it relies on.

3.2 Quantum Computing Foundations

3.2.1 Qubits and Quantum States

The fundamental unit of quantum information is the **qubit** (short for quantum bit). While a classical bit can hold can have two values, 0 or 1, a qubit can exist in a state called "superposition" of both basis states simultaneously. Using the Dirac "bra-ket" notation, the general state of a qubit is a unit vector in the two-dimensional complex Hilbert space $\mathbb{C}^2$, written as:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, \quad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1,$$

where $|0\rangle$ and $|1\rangle$ are the two computational basis states.

3.2.2 Measurement and State Collapse

Measuring a qubit in a chosen base collapses its state into one of the basis vectors. If the qubit is in the state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ and we measure it in the computational basis $\{|0\rangle, |1\rangle\}$, the probability that the outcome will be 0 is $|\alpha|^2$, and for the outcome 1 it will be $|\beta|^2$ (the Born rule). After the measurement, the qubit is in the base-state corresponding to the observed outcome, while the original amplitudes α, β cannot be recovered.

If a qubit is prepared in one basis and then measured in a different basis, the outcome is essentially random and offers no additional information about the original state. For a concrete example, let's consider a qubit prepared in the state $|0\rangle$ (a Z-basis state) and then measured in the diagonal X-basis. Rewriting $|0\rangle$ in the X-basis gives

$$|0\rangle = \frac{1}{\sqrt{2}}(|+\rangle + |-\rangle),$$

so the measurement returns $|+\rangle$ with probability $\frac{1}{2}$ and $|-\rangle$ with probability $\frac{1}{2}$. The outcome carries no information about the original Z-basis preparation, and the qubit collapses irreversibly into the measured X-basis state. The reverse situation (i.e. X-basis preparation, Z-basis measurement) is analogous.

3.2.3 Hadamard Gate and the Diagonal Basis

In addition to the computational Z-basis $\{|0\rangle, |1\rangle\}$, BB84 also uses the diagonal basis (X-basis) $\{|+\rangle, |-\rangle\}$, defined by:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}.$$

The two bases are incompatible: a qubit prepared as $|0\rangle$ or $|1\rangle$ gives a random outcome when measured in $\{|+\rangle, |-\rangle\}$ and vice-versa.

The transformation between the two bases is performed by the **Hadamard gate** H :

$$H|0\rangle = |+\rangle, \quad H|1\rangle = |-\rangle,$$

described by the following matrix:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

A quantum gate transforms a qubit state, similar to a classical logic gate. As seen above, the Hadamard gate maps the computational basis into the diagonal basis

In the **Sequire** implementation, the Hadamard gate is applied in Qiskit via `'qc.h(0)'` when Alice encodes a bit in the X -basis (diagonal). Bob applies the same Hadamard gate before a standard measurement in the computational basis, which is the equivalent to measuring directly in the diagonal basis.

3.2.4 Pauli Gates

The **Pauli gates** (X , Y , Z) are the single-qubit operations corresponding to the Pauli matrices:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

The X gate is the quantum equivalent of the classical NOT gate. The Z gate leaves $|0\rangle$ unchanged and flips the sign of $|1\rangle$, defining the computational basis. The Y gate combines both effects. In the **Sequire** BB84 implementation, the X gate is applied explicitly via `'qc.x(0)'` when Alice encodes a 1 bit in the Z -basis, while the Z basis itself serves as the default computational measurement basis (no Z -gate is applied; instead, measurement is performed directly in this basis).

3.3 The No-Cloning Theorem

The **no-cloning theorem** (Wootters and Zurek, 1982 [6]) states that no quantum operation can produce an exact copy of an arbitrary quantum state. A short algebraic argument illustrates this: if a unitary cloning operation existed, then for any two input states $|\psi\rangle$ and $|\phi\rangle$ the inner product would have to satisfy $\langle\psi|\phi\rangle = \langle\psi|\phi\rangle^2$, an identity that holds only when the states are either orthogonal or identical.

The consequence for QKD is direct. Eve cannot intercept a qubit, keep a perfect copy for herself and forward the original to Bob, unchanged. Any attempt to extract information from the channel forces her to interact with the qubit, and that interaction disturbs it, as the next section shows.

3.4 Heisenberg's Uncertainty Principle in QKD

The BB84 protocol rests on **Heisenberg's uncertainty principle**, which states that it is impossible to simultaneously know, with full precision, the outcome of measurements in two incompatible bases. This is a purely quantum-mechanical phenomenon, because in classical physics, any observable can in principle be measured arbitrarily precisely without disturbing the others. Applied to the Z -basis and the X -basis, this means that

if Alice prepares a qubit in one basis and Eve measures it in the other, the outcome Eve will read is purely random, and the act of measuring collapses the qubit into a state that no longer encodes Alice's original bit. This is the mechanism by which eavesdropping is detected in BB84.

3.5 Quantum Entanglement

An indispensable principle of quantum mechanics that QKD can use is **quantum entanglement**: two particles can be prepared in a common state such that measuring one instantly fixes the outcome of the other, no matter how far apart they are. The correlation between the two outcomes is stronger than anything classical pre-arrangement can ever produce, and this gap is what the **E91** protocol (section 3.13.3) exploits to derive a shared key and to detect an eavesdropper at the same time.

3.6 Quantum Bit Error Rate (QBER)

The **Quantum Bit Error Rate** (QBER) is defined as the fraction of sifted key bits on which Alice and Bob "disagree":

$$Q = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[a_i \neq b_i],$$

where N is the length of the sifted key, a_i, b_i are the corresponding bits held by Alice and Bob after the sifting procedure and $\mathbf{1}[\dots]$ is the indicator function (equal to 1 when the condition is true and 0 otherwise).

In a noise-free channel without eavesdropping, every sifted bit satisfies $b_i = a_i$, so $QBER = 0$. Real channels and detectors add noise, which raises $QBER$ by a small, bounded amount. An active intercept-resend attack raises $QBER$ significantly above any noise floor. Specifically:

- **No Eve, no noise:** $QBER \approx 0$.
- **Weak Eve:** (i.e. Eve intercepts a fraction p of qubits): each intercepted qubit introduces an error with probability $\frac{1}{4}$ (wrong basis with the probability $\frac{1}{2}$, error given wrong basis with the probability $\frac{1}{2}$), so $QBER \approx \frac{p}{4}$. ($p = 0.30$), this gives $QBER \approx 7.5\%$.
- **Strong Eve:** ($p = 1.0$, full intercept-resend): $QBER \approx 25\%$.

Sequire uses a security threshold of 11%, consistent with the standard BB84 security analysis [8]: if $QBER \geq 11\%$, the protocol is aborted and no key will be output. This threshold ensures that any remaining information Eve holds about the key can be made exponentially small with the help of privacy amplification.

3.7 Error Correction & Privacy Amplification

After sifting and the QBER verification, now Alice and Bob have two strings of bits which should match on most of the positions, but usually they do not match exactly: the noise within the channel produces a non-zero QBER even when there is no intercepting. The last two steps of BB84 reconcile the two keys into an identical key and then compress the result in order to eliminate any potential knowledge Eve may have gained

3.8 The BB84 Protocol

As introduced in chapter 1, BB84 [7] is the first QKD protocol, proposed by Bennett and Brassard in 1984, and involves the three actors **Alice** (sender), **Bob** (receiver), and **Eve** (potential eavesdropper). What distinguishes BB84 from classical key-exchanging methods is the environment itself: each key bit is encoded into the polarisation state of a single qubit, so any attempt by Eve to measure or copy said qubit disturbs it in a way the legitimate parties can detect by comparing a small subset of their results. The protocol produces a common secret key between Alice and Bob in six steps. The first three establish a raw shared sequence of bits over a quantum channel; the last three transform it into a final secret key using a classical public channel. The steps are:

Step 1: Qubit preparation (Alice). Alice generates a random bit string $\mathbf{a} = (a_1, \dots, a_n)$ and an independent random basis string $\mathbf{x} = (x_1, \dots, x_n)$, where each x is either X or Z . She encodes each bit a_i into a qubit according to the table below and sends its the qubits one by one over the quantum channel.

Bit	Basis	State	Description
0	Z (rectilinear)	$ 0\rangle$	spin-up / horizontal polarisation
1	Z (rectilinear)	$ 1\rangle$	spin-down / vertical polarisation
0	X (diagonal)	$ +\rangle = \frac{1}{\sqrt{2}}(0\rangle + 1\rangle)$	$+45^\circ$ polarisation
1	X (diagonal)	$ -\rangle = \frac{1}{\sqrt{2}}(0\rangle - 1\rangle)$	-45° polarisation

In practice, a qubit is built like a physical system with two layers. The most common implementations are the spin of the electron ($|0\rangle = \text{spin-up}$, $|1\rangle = \text{spin-down}$) and the photon polarization ($|0\rangle = \text{horizontal}$, $|1\rangle = \text{vertical}$); most of the times, BB84 uses photons, while **Sequire** uses the abstract qubit model from Qiskit, without a specific physical implementation.

Step 2: Qubit measurement (Bob). Bob independently chooses a random basis string $\mathbf{y} = (y_1, \dots, y_n)$ and measures each received qubit in the corresponding basis,

recording his outcomes as $\mathbf{b} = (b_1, \dots, b_n)$.

Step 3: Basis sifting. Alice and Bob communicate their basis choices $\mathbf{x}$ and $\mathbf{y}$ over a classical public channel (no bit values are revealed). They retain only the positions where $x_i = y_i$ (on average, 50% of all transmitted qubits), and then the retained bits form the **sifted key**. The term "sifting" expresses the idea of filtering: Alice and Bob "sift" all the positions in which the bases have not matched, keeping those positions.

Step 4: Error rate estimation (QBER). Alice and Bob sacrifice a random subset of their sifted key to estimate the Quantum Bit Error Rate (QBER). If it exceeds a security threshold, they abort the protocol (more detailed in the below section 2.6).

Step 5: Error correction. Alice and Bob apply an error-correction protocol over the public channel to reconcile the remaining bits of their sifted keys, so that the two keys become identical. (more in section 3.7).

Step 6: Privacy amplification. To account for any partial information Eve may have obtained, Alice and Bob apply a universal hash function, SHA-256 to compress their reconciled key into a shorter final key, whose correlation with anything Eve would know is negligible. (more in section 3.7).

3.8.1 Error Correction

In **error correction** (also called information reconciliation), Alice and Bob communicate over the public classical channel to make their two keys match.

Sequire uses a simple **parity-block scheme**: the sifted key is divided into consecutive blocks of 4 bits. For each block, Alice announces the parity (XOR of the 4 bits) on the public channel, and Bob compares it with his own. If both parties agree, they keep the block, but if they disagree, the block contains at least one error and is discarded by both of them. This scheme is simple to analyze, but inefficient. Why? Because it sacrifices an entire block for any single bit error.

3.8.2 Privacy Amplification

After error correction, Alice and Bob share an identical key, but Eve may hold partial information about it: she observed every parity announcement on the public channel during the error correction phase, and she may have obtained information from any qubits she intercepted before the QBER check revealed her presence. **Privacy amplification** compresses the reconciled key with a hash function (specifically SHA-256, producing a key that's 256 bits long, but truncating it to 128 bits as the *final key*),

reducing Eve's information about the output key to a negligible quantity, regardless of how much she learned about the input [11].

3.9 The CASCADE Information-Reconciliation Algorithm

While the parity-block scheme discards an entire block on any disagreement regarding the bits, **CASCADE** locates and flips each incorrect bit individually via binary search on parities, keeping the rest of the key intact. The algorithm runs in several passes. The first pass uses a block size B_1 , chosen from the estimated QBER, and each next pass doubles it.

$$B_1 \approx \left\lceil \frac{0.73}{Q} \right\rceil, \quad B_{i+1} = 2 B_i.$$

At the start of a pass, the key is randomly permuted and split into blocks of size B_1 . Alice announces the parity of the block on the public channel, and for every block whose parity does not match with Bob's, the two apply a binary search: it splits the block into two halves, compare their parity, then it recursively descends into the half where the parities differ and it stops when the incorrect bit is identified and flipped.

The key step is the **back-correction** (i.e. retroactive correction). Flipping a bit in i -th pass might invalidate the parity of a block from a previous pass that contained the same bit, revealing a second error that was hidden until now. That's why CASCADE recursively revisits earlier blocks. After several passes, it converges with high probability, leaking a number of parity bits, number that is close to the **Shannon lower bound** (the information-theoretic minimum number of parity bits required for reconciliation at the given QBER) [10]. That total leak of information is the exact quantity that will be subtracted during privacy amplification in order to obtain the final secret key length.

3.10 Optical Fibre Channel Attenuation

Real life implementations of the BB84 protocol carry the qubits as photons through optical fibre. On long distances, the dominant limit is not the noise, but the **photon attenuation** within the channel. For standard optical fibre, which is 1550 nanometers in wavelength, the survival probability of a transmitted photon follows the **Beer-Lambert** law:

$$\eta(L) = 10^{-\alpha L/10}, \quad \alpha \approx 0.2 \text{ dB/km},$$

where $\eta(L)$ represents the probability that a photon will survive over a distance of L kilometres and α is the attenuation coefficient of the optic fibre (took from the

ITU-T G.652 [13] standard). This means that only $\sim 10\%$ of photons survive a 50km long link and $\sim 1\%$ do so on a 100km. This exponential decay sets the fundamental rate-distance limit for direct BB84 transmission and is one of the main reasons real implementations rely on weak coherent pulses with decoy-state monitoring, as discussed in the next section.

3.11 Decoy-State Method & Photon Number Splitting Attack

QKD systems do not have single-photon sources. Instead, they use attenuated lasers, otherwise called **weak coherent pulses** (WCP). A WCP with mean photon number μ emits n photons per pulse with Poisson probability:

$$P(n | \mu) = e^{-\mu} \frac{\mu^n}{n!},$$

where $P(n | \mu)$ is the probability a pulse will contain exactly n photons given an intensity μ . For $\mu < 1$ most pulses contain either no photons or one photon, but a small fraction of pulses contains $n \geq 2$ photons. These multi-photon pulses make the PNS attack possible. Overall, the PNS attack is most dangerous over long distances, where channel loss suppresses the legitimate single-photon signal while multi-photon pulses pass through unaffected.

The **Decoy-State Method** [15, 16] defends against PNS attacks by having Alice randomly alternate between a signal intensity μ (used for the key) and one or more decoy intensities ν , used for monitoring. Thus, Eve cannot distinguish in advance the signal pulses from the decoy pulses, so any selective suppression of single-photon pulses shows up as a discrepancy between the gains of the two intensities. The **gain** at intensity μ is the total detection rate $Q_\mu = \sum_n P(n | \mu) Y_n$, where Y_n is the conditioned probability that Bob will detect a pulse containing exactly n photons. Combining the two "gains" measured at the two intensities, the analysis will produce a lower bound (limit) on the single-photon yield Y_1 :

$$Y_1^L \geq \frac{\mu}{\mu\nu - \nu^2} \left(Q_\nu e^\nu - Q_\mu e^\mu \frac{\nu^2}{\mu^2} - \frac{\mu^2 - \nu^2}{\mu^2} Y_0 \right),$$

where μ and ν are the signal and decoy intensities respectively, Q_μ and Q_ν are the measured gains at each intensity, and Y_0 is the background noise of the so-called **dark counts** (i.e. detections triggered by the detector noise). Y_1^L is the central quantity: it lower-bounds the rate at which genuine single-photon events reach Bob, and the secure key rate is then computed from Y_1^L via the **GLLP formula** [14]. Sequire's decoy-state panel renders Q_μ , Q_ν , Y_1^L and the resulting key rate live, so the user can watch the single-photon contribution being recovered from the WCP statistics in real time.

This formula is named after Daniel Gottesman, Hoi-Kwong Lo, Norbert Lütkenhaus, and John Preskill (2004), who proved the asymptotic security bound for BB84 with imperfect devices (weak coherent pulse sources, lossy channels, and noisy detectors). It takes the single-photon yield Y_1^L , the single-photon error rate, and the measured QBER as inputs and returns the number of secure key bits Alice and Bob can extract per pulse after subtracting both the error-correction leakage and the privacy-amplification overhead.

3.12 Machine Learning for QKD Security Monitoring

The QBER threshold check catches any adversary that exceeds the 11% abort threshold, but it is blind by design to an attacker that purposefully stays below it (i.e. a smart Eve, see section 7.3). To detect this case, **Sequire** adds a complementary layer of analysis: a binary classifier that takes the output of one BB84 run and returns the estimated probability that Eve was present.

Eavesdropping detection in QKD is a manner of binary classification problem: given the output of a BB84 run, decide whether Eve was present. Many machine learning algorithms can solve it: classical methods like **logistic regression**, **support vector machines**, or **gradient-boosted trees** work on fixed-length feature vectors, while deep models such as feed-forward networks, one-dimensional convolutional networks, and transformer-based sequence models learn patterns directly from the per-qubit sequence. **Sequire** implements one algorithm from each family: **Random Forest** for fast, interpretable verdicts from post-run features, and an LSTM that reads the per-qubit sequence position by position.

The **Random Forest** [17] works by training many decision trees on different random subsets of the data and combining their votes into a single prediction. **Sequire** uses it for two practical reasons. Firstly, it reports which inputs were most influential in each decision through feature-importance scores. Secondly, it works well on small datasets, the kinds that accumulate during a single user session, without needing a large precomputed training set. In **Sequire**, the input is a vector of six numbers measured after the run finishes: the QBER, the sifting efficiency, the configured channel distance, the depolarising-noise probability, the measurement-error probability, and the sifted-key length. The output is the estimated probability that Eve was present, $\hat{P}(\text{Eve}) \in [0, 1]$, together with a list of how much each input contributed to that probability. The model is retrained from the user's own simulation history, which is stored locally, so it adapts to the runs the user has actually performed rather than to a fixed external dataset.

The **Long Short-Term Memory** (LSTM) network [18] is a type of neural network designed for sequences. It reads inputs one at a time, in order, and keeps an internal

memory that lets it remember what it saw earlier in the sequence. **Sequire** uses it for the per-qubit task because the intercept-resend attack leaves a temporal pattern: errors cluster on basis-matched positions where Eve picked the wrong basis, while basis-mismatched positions stay random regardless of whether Eve is there. The Random Forest cannot see this pattern because it only receives summary statistics, whereas the LSTM reads the qubits one at a time and can pick up the pattern directly. At each position it reads Alice's basis, Alice's bit, Bob's basis, Bob's outcome, and the basis-matched flag, then updates its internal memory. The model is trained on a synthetic dataset of labelled runs (with Eve present or absent at different interception rates), and after training it returns a single probability that Eve was present in the current run, $P(\text{Eve} \mid \text{sequence})$.

Together the two detectors form a layered defence: the Random Forest provides a fast, interpretable verdict from coarse features, while the LSTM provides a sharper signal in the sub-threshold regime where the Random Forest would miss the attack. The empirical performance of both models is reported in Section 8.8.

3.13 Alternative QKD Protocols

Sequire also implements another three QKD protocols, which are **B92**, **SARG04** and **E91**, the first two being closely related to BB84. In short, B92 simplifies its encoding, while SARG04 modifies its public announcement. E91 takes a completely different route, deriving its security from *quantum entanglement* rather than disturbance under measurement

3.13.1 B92 (Bennett, 1992)

The B92 protocol [19] was proposed by none other than **Charles Bennett**, the first half of the pair that developed the BB84 protocol. It proves that we can make QKD only using two quantum states, one for each base. It simplifies BB84 to just two non-orthogonal states: Alice sends $|0\rangle$ for bit 0 and $|+\rangle$ for bit 1. Bob measures randomly in the Z or X bases and keeps only conclusive outcomes (i.e. those that are incompatible with one of Alice's two possible states). Its disadvantages are that about 25% of qubits survive sifting (compared to 50% in BB84) and the protocol is more vulnerable to the noise within the optical fibre and less efficient in practice than BB84, thus being rarely used in modern-day commercial systems.

3.13.2 SARG04 (Scarani-Acín-Ribordy-Gisin, 2004)

SARG04 [20] was created to counter the PNS attacks, those where Eve "steals" a photon from the pulses that mistakenly contain more than one photon. SARG04 uses the

same four qubit states as BB84 but it changes the manner of the public announcement: instead of revealing her base, Alice announces an unordered pair of non-orthogonal states, one of which is the one she sent. Bob retains the bit only if his outcome is consistent with one of the two. Since Eve cannot tell which state was sent without measuring first, SARG04 resists the PNS attack more effectively than BB84 does, at the cost of $\approx 25\%$ sifting efficiency.

3.13.3 E91 (Ekert, 1991) and the CHSH Bell Test

While BB84 derives its security from the no-cloning theorem and measurement disturbance, E91 [21] derives it from **quantum entanglement**. A central source emits pairs of photons in the entangled **Bell state** $|\Phi^+\rangle$, one of four maximally entangled two-qubit states named after John Bell:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle),$$

which couples the two particles such that a measurement on one instantly fixes the outcome of the other. The source sends one photon to Alice and one to Bob, both measuring their photon at a randomly chosen orientation (basis) from a fixed set of three:

$$\begin{aligned} \text{Alice: } a_1 &= 0^\circ, a_2 = 45^\circ, a_3 = 90^\circ, \\ \text{Bob: } b_1 &= 45^\circ, b_2 = 90^\circ, b_3 = 135^\circ. \end{aligned}$$

Within the E91 protocol, angles are strategically chosen at intervals of 45 degrees, because at differences of 45° and 135° , the trigonometric functions *sin* and *cos* that govern entangled photons reach values of $\pm \frac{\sqrt{2}}{2} \approx \pm 0.707$. When their bases match ($a_2 = b_1 = 45^\circ$ or $a_3 = b_2 = 90^\circ$), the outcomes are perfectly correlated and form the raw key. When the bases differ, the measurement results fuel the **CHSH correlator** S (Clauser, Horne, Shimony, Holt, 1969), a numerical version of the Bell inequality that distinguishes the classical pre-established correlations from the actually quantum-entangled ones:

$$S = E(a, b) - E(a, b') + E(a', b) + E(a', b'),$$

where a, a' are two of Alice's orientations, b, b' are two of Bob's, and $E(\theta_a, \theta_b) = P(\text{same} \mid \theta_a, \theta_b) - P(\text{different} \mid \theta_a, \theta_b)$ is the correlation between their outcomes at those orientations.

Concretely, S is bounded at $|S| \leq 2$ for any classical correlations and can reach $|S| = 2\sqrt{2}$ (the Tsirelson limit) for ideal quantum-entanglement. Eavesdropping breaks the entanglement and drives $|S|$ towards the classical bound, but under strong Eve, S

collapses to almost 0. In the end, **Sequire** computes S after every E91 run and plots it between the two bounds.

3.14 Finite Key Security Bounds

The standard BB84 security proof assumes an infinite key length. In practice, any real run uses a finite block of n sifted bits and the measured QBER ($\hat{Q}$) fluctuates by an amount of order $1/\sqrt{n}$. A rigorous analysis with finite keys must keep this fluctuation in mind.

The reference analysis is given by **Marco Tomamichel** and his collaborators [23], reference that is the standard on tight finite key bounds in QKD. They correct the measured QBER through a Hoeffding bound:

$$Q_{\text{upper}} = \hat{Q} + \delta_{\text{PE}}, \quad \delta_{\text{PE}} = \sqrt{\frac{\ln(2/\varepsilon_{\text{PE}})}{2n}},$$

where δ_{PE} is a small statistical penalty (or safety margin, of order $1/\sqrt{n}$) added to $\hat{Q}$ to obtain a worst-case scenario QBER Q_{upper} , and ε_{PE} is the chosen confidence parameter. The resulting key length is bounded by:

$$\ell \leq n \left[1 - H_2(Q_{\text{upper}}) \right] - n f_{\text{EC}} H_2(\hat{Q}) - \log_2 \frac{2}{\varepsilon_{\text{corr}}} - 2 \log_2 \frac{1}{2\varepsilon_{\text{PA}}},$$

where H_2 is the binary entropy, $f_{\text{EC}} \approx 1.16$ is the CASCADE efficiency, and the four terms are (in order) the asymptotic key gain, the error-correction leakage, and the correctness and privacy-amplification overheads, controlled by the small failure probabilities $\varepsilon_{\text{corr}}$ and ε_{PA} .

Sequire's 'Finite Key' panel evaluates this bound live and flags whether the current block size yields a positive key length or not.

4 Existing QKD Simulators and Related Work

Several software platforms exist for simulating QKD. This chapter surveys the most widely used tools, evaluates their scope and limitations and positions **Sequire** relative to the state of the art.

4.1 NetSquid

NetSquid [25] (Network Simulator for Quantum Information using Discrete events) is a high-fidelity, discrete-event simulator for quantum networks. Its primary design goal is performance characterisation of quantum repeater networks, entanglement distribution protocols, and quantum memory systems, not QKD education. It models physical processes (photon propagation, decoherence, detector timing jitter, dark counts) at a component level of detail that greatly exceeds what is needed for demonstrating BB84. As a consequence, NetSquid requires substantial domain expertise to use, has no browser-accessible interface, and is not suited to interactive classroom demonstration. It also does not integrate with real IBM Quantum hardware.

4.2 QuNetSim and SimulaQron

QuNetSim [26] and SimulaQron [27] are quantum-network simulation frameworks that provide an abstraction layer for writing distributed quantum protocols in Python. Both can simulate QKD as a special case but are designed as generic network-layer toolkits rather than QKD-specific platforms. They expose low-level primitives (qubit send/receive, teleportation, entanglement swapping) rather than a pedagogically structured BB84 pipeline. Neither provides real-time visualisation, a web interface, or integration with cloud quantum hardware.

4.3 Qiskit Textbook Demos

IBM's Qiskit learning platform includes interactive Jupyter-notebook demonstrations of BB84 that use Qiskit circuits and Aer simulation. These notebooks are closest in spirit to **Sequire**: they use the same Qiskit circuit model, support noise injection, and are intended for students. They are however static notebooks rather than a deployable web application, and do not persist results, stream events live, integrate with hardware via a polished interface, or offer adversarial simulation.

4.4 Positioning of Sequire

The table below summarises the comparison across the dimensions most relevant to **Sequire**'s design goals.

Feature	Sequire	NetSquid	QuNetSim	Qiskit nb.
Real IBM hardware execution	✓	—	—	—
Real-time per-qubit visualisation	✓	—	—	—
Decoy-state / PNS / finite key	✓	—	—	—
ML eavesdropper detection	✓	—	—	—
Gamified Challenge Mode	✓	—	—	—
Component-level physical fidelity	—	✓	—	—
Network-layer protocol primitives	—	—	✓	—
Official IBM curriculum integration	—	—	—	✓

Sequire's contribution lies in combining three things at once: research-level analytical depth (decoy-state bounds, finite key security, LSTM detection, real-hardware execution), a browser-accessible interactive interface, and a gamified learning mode. **NetSquid** covers the physics in greater depth, **QuNetSim** offers broader network primitives, and the **Qiskit** notebooks carry IBM's official curriculum status.

5 Technologies Used

This chapter describes the core technologies that underpin the **Sequire** platform, covering both the backend computation stack and the frontend presentation layer.

5.1 Python 3.11

The backend is written in Python 3.11. Python was chosen for its mature ecosystem of quantum-computing libraries (Qiskit, PyTorch), scientific computing packages (NumPy), and web frameworks (FastAPI). The 3.11 release specifically was selected for its 10–25% speed improvement over Python 3.9 in CPU-bound loops, which is measurable during the per-qubit simulation loop at larger qubit counts.

5.2 Qiskit & Qiskit-Aer

Qiskit [24] is IBM's open-source SDK for quantum computing. **Sequire** uses it to construct per-qubit BB84 circuits: Alice's encoding gate sequence (optional X gate for bit 1, optional H gate for the X -basis) and Bob's measurement in the chosen basis. The circuit for a single qubit exchange is a one-qubit circuit of depth at most 3, kept intentionally shallow to minimise transpilation overhead when dispatching to real hardware.

Qiskit-Aer is the high-performance simulator backend. It supports both ideal statevector simulation (used for the zero-noise baseline) and noisy simulation via the NoiseModel API. **Sequire** builds a noise model with depolarising errors on the X and H gates and a Pauli bit-flip channel on measurement, parameterised by the user-controlled 'depolarizing_prob' and 'measurement_error_prob' inputs.

5.3 Qiskit IBM Runtime

For real-hardware execution, **Sequire** integrates with the **Qiskit IBM Runtime** SamplerV2 API. Individual BB84 circuits are batched into a single SamplerV2 job, dispatched asynchronously to a configurable IBM Quantum backend (e.g. 'ibm_fez', 'ibm_marrakesh', 'ibm_kingston' or any other device accessible to the configured IBM). Results are streamed to the frontend over the open WebSocket connection as soon as the job completes. Access credentials are stored in a server-side '.env' file that is never committed to version control.

5.4 FastAPI

FastAPI was mainly chosen for three reasons that mattered to my use case. First, its native `async/await` support made it straightforward to stream per-qubit events over a WebSocket without blocking the event loop on the Qiskit simulation calls (achieved via `asyncio.sleep(0)` yield points). Second, its automatic OpenAPI documentation (available at `/docs`) documents all REST endpoints without additional effort. Third, its Pydantic integration provides input validation and serialisation for all request and response models.

Sequire's FastAPI application exposes two types of endpoints: REST endpoints (simulation history, ML training, authentication, challenge progress) under `/api/v1/`, and a single WebSocket endpoint at `/ws/simulate/{session_id}` that drives the live simulation stream.

5.5 WebSockets for Real-Time Streaming

The WebSocket protocol provides a full-duplex, persistent TCP connection over which the server can push events to the client without polling. In **Sequire**, the frontend opens a WebSocket to `/ws/simulate/{uuid}`, sends the simulation configuration as a JSON object, and then receives a stream of `type: "qubit"` events (one per simulated qubit, carrying Alice's basis and bit, Bob's basis and result, and the Eve-intercept flag) followed by a single `type: "result"` event containing the full summary. This streaming architecture is the technical foundation of the PhotonGrid animation and the progressive QBER update.

5.6 SQLite and SQLAlchemy

Simulation runs, user accounts, password-reset tokens, and Challenge Mode attempts are persisted in a **SQLite** database (`'sequire.db'`) via the **SQLAlchemy** ORM. SQLite was selected over a client-server database (e.g. PostgreSQL) because **Sequire** is designed for single-host deployment; the operational write rate (one INSERT per simulation run) is well within SQLite's concurrency limits. SQLAlchemy provides declarative model definitions and a session-based transaction API; schema migrations are handled via idempotent `"ALTER TABLE"` statements executed at startup, which add new columns if absent and silently skip them otherwise.

5.7 PyTorch

PyTorch is used for the LSTM eavesdropper detector (section 3.12). The CPU build (no CUDA required) is installed as a backend dependency. The LSTM, linear classifier, and

training loop are implemented in 'backend/ml/lstm_detector.py'; the trained weights are saved to 'lstm_weights.pt' at the end of the first training run and loaded from disk on subsequent server starts. The entire training cycle (1 200 synthetic sequences, 60 epochs) completes in under 30 seconds on a modern CPU.

5.8 React 19 and Vite

The frontend is a **React 19** single-page application bundled with **Vite**. React's component model allows each visualisation panel (PhotonGrid, DecoyStatePanel, FiniteKeyPanel, LstmDetectionPanel, etc.) to be a self-contained, props-driven component that re-renders only when its data changes. Vite provides sub-second hot-module replacement during development and a highly optimised production bundle.

Client-side routing between the main Dashboard (/) and the Challenge Mode page (/challenge) is provided by **React Router DOM v7**, which intercepts URL changes and renders the appropriate page component without a full-page reload.

5.9 Tailwind CSS

Tailwind CSS is a utility-first CSS framework that generates a minimal stylesheet containing only the classes actually used in the project. **Sequire** uses Tailwind exclusively for styling: all component layout, colour, spacing, and responsive behaviour is expressed via inline class names. The dark-mode design (gray-950 background, violet accent) is achieved with Tailwind's colour palette without any custom CSS files.

5.10 Recharts

The QBER historical chart and the key-rate-vs-distance curve are rendered with **Recharts**, a composable charting library built on top of D3 and React. It was chosen for its declarative API and out-of-the-box responsiveness, which eliminates the need for manual SVG manipulation.

5.11 Authentication Stack

The authentication subsystem uses:

- **bcrypt** for password hashing (work factor 12);
- **python-jose** for JWT access and refresh token signing (HS256, configurable secret);
- **Starlette 'SessionMiddleware'** for the OAuth `state` parameter across the Google redirect;
- **Authlib** for the Google OAuth 2.0 flow (authorisation-code with PKCE);

- **HttpOnly, SameSite=Lax** cookie delivery of the access token, combined with a CSRF double-submit cookie for state-changing requests.

6 Application Architecture

Sequire follows a classic three-tier web architecture: a **React SPA** frontend served by Vite (development) or a static file server (production), a **FastAPI** backend that exposes REST and WebSocket endpoints, and a **SQLite** database for persistence. The backend communicates outwards to the IBM Quantum Runtime API for hardware execution. The frontend additionally performs an unauthenticated geolocation lookup to `ipapi.co` for the session metadata widget, lookup that does not transit the backend.

6.1 High-Level System Overview

The deployment topology is single-host: both the frontend (port 5173 in development, served from `/` by the backend's static-file middleware in production) and the backend (port 8000) run on the same machine. Vite's development proxy forwards all `/api/v1/` and `/ws/` requests to the FastAPI process, so the browser sees a single origin with no cross-origin restrictions. A `.env` file at the backend root holds all secrets (JWT key, IBM token, Google OAuth credentials) and is excluded from version control.

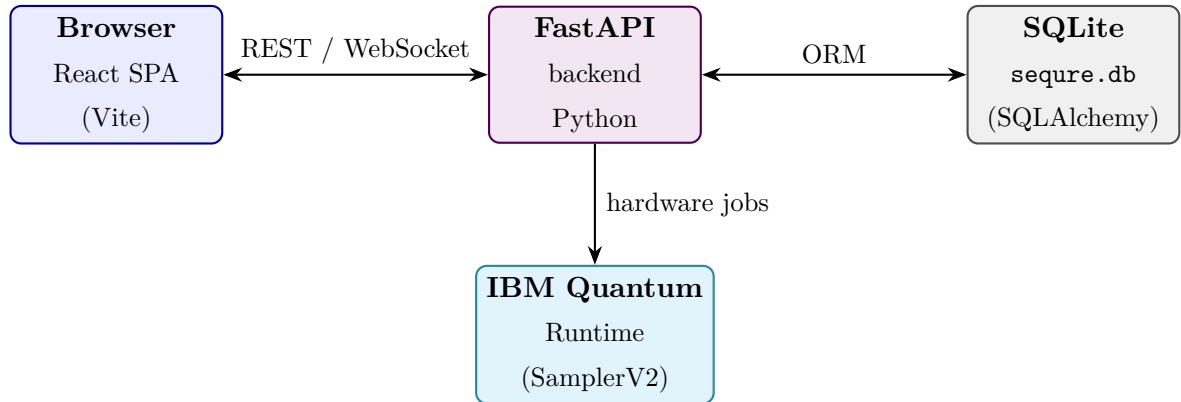


Figure 1 – Sequire deployment topology. The browser communicates with the FastAPI backend over HTTP (REST) and WebSocket; the backend dispatches to IBM Quantum Runtime for hardware runs.

6.2 Backend Component Structure

The backend Python package is organised into the following top-level modules, each with a single well-defined responsibility:

api/ FastAPI routers. `routes.py` handles REST (history, ML train/status, key-rate curve); `'websocket_routes.py'` contains the main simulation loop; `'challenge_routes.py'` exposes the Challenge Mode endpoints.

quantum_logic/ Protocol-independent quantum circuit construction (`'bb84_circuit.py'`), noise model (`'noise_model.py'`), channel attenuation (`channel_model.py`), WCP/decoy

simulation and PNS attack (`'decoy_state.py'`), finite key analysis (`'finite_key_analysis.py'`), Smart Eve controller (`'smart_eve.py'`), and the alternative-protocol plug-ins (`protocols/`).

classical_processing/ Sifting (`sifting.py`), QBER calculation (`qber.py`), parity-block error correction (`error_correction.py`), CASCADE reconciliation (`cascade.py`), and privacy amplification (`privacy_amplification.py`).

ml/ Random Forest classifier (`eavesdrop_detector.py`) and LSTM detector (`lstm_detector.py`).

challenge/ Mission catalogue (`mission_catalog.py`), parameter instantiator (`instantiator.py`), Detective and Engineer graders (`grader.py`), and persistence helpers (`persistence.py`).

auth/ JWT security (`security.py`), FastAPI dependencies (`dependencies.py`), OAuth flow (`oauth.py`), and route handlers (`routes.py`).

database/ SQLAlchemy models (`models.py`) and database initialisation (`db.py`).

6.3 Frontend Component Structure

The frontend is structured around a single App component that provides routing (React Router) and two context providers (AuthContext, ChallengeContext):

Dashboard (/) The main simulation page. Contains `SimulationControls` (left sidebar), the `PhotonGrid`, and a column of result panels: `SimulationSummary`, `DecoyStatePanel`, `PnsAttackPanel`, `FiniteKeyPanel`, `LstmDetectionPanel`, `SmartEvePanel`, `BellTestPanel`, `MLDetectionPanel`, `QBERChart`, `KeyRateChart`, `FunFactPanel`, and `EducationalPanel`.

ChallengeMode (/challenge) The gamified learning page. Contains `LevelGrid`, `MissionBriefing`, `MissionResultModal`, and `LeaderboardPanel`.

State for the active simulation is managed by the `useSimulationSocket` hook, which owns the Websocket lifecycle and exposes `result` (qubit-level accumulator) and `summary` (final result object) to all panels. Challenge Mode state is managed by `useChallengeSession`, a finite state machine with states `idle`, `briefing`, `running`, `awaiting_answer`, and `result`.

6.4 Database Schema

The SQLite database contains five tables:

users Authenticated user accounts. Columns: `id`, `email` (unique), `full_name`, `password_hash` (nullable for OAuth-only accounts), `google_id`, `avatar_url`, `is_email_verified`, `created_at`, `last_login_at`.

- simulation_runs** One row per completed simulation. Columns: `id`, `user_id` (nullable FK, SET NULL on delete), `n_qubits`, `depolarizing_prob`, `measurement_error_prob`, `eve_intercept` (bool), `sifted_key_length`, `qber`, `is_secure`, `final_key`, `channel_distance`, `created_at`.
- password_reset_tokens** Short-lived tokens for the email-based password-reset flow. Columns: `id`, `user_id` (CASCADE DELETE), `token_hash` (unique, bcrypt), `created_at`, `expires_at`, `used_at`.
- mission_attempts** One row per Challenge Mode attempt. Columns: `id`, `user_id`, `template_id` (FK to catalogue, string), `level_number`, `instantiated_params` (JSON), `user_answer` (JSON), `correct`, `score`, `feedback` (JSON), `simulation_run_id` (nullable FK), `started_at`, `completed_at`.
- user_progress** Aggregated per-user Challenge Mode statistics. Primary key is `user_id` (one row per user). Columns: `levels_unlocked`, `levels_completed`, `total_xp`, `best_streak`, `current_streak`, `updated_at`.

Schema migrations (adding new columns to existing tables) are applied at startup via idempotent `ALTER TABLE` statements that silently succeed if the column is already present, ensuring forward compatibility across deployments without a dedicated migration tool.

6.5 Request and Event Flow

A complete simulation cycle proceeds as follows:

1. The user configures parameters in 'SimulationControls' and presses *Run Simulation*.
2. The 'useSimulationSocket' hook opens a WebSocket to `/ws/simulate/{uuid}` and sends the configuration JSON.
3. The FastAPI handler authenticates the optional session cookie, reads the configuration, and dispatches to the appropriate simulation path ('_run_bb84_path' or '_run_alt_protocol').
4. For each qubit, the handler simulates the quantum circuit, applies Eve's interception logic, detector imperfections, and channel loss, then emits a `type: "qubit"` event via the WebSocket session manager.
5. The frontend accumulates qubit events, updating `result` state in real time; the 'PhotonGrid' re-renders on each event.
6. After all qubits, the handler runs classical post-processing (sifting, QBER, EC,

PA, decoy analysis, finite key bounds, ML inference), persists the 'SimulationRun' row, and emits a `type: "result"` event with the full summary.

7. The frontend populates all result panels from the summary object.

7 Implementation and Features

This chapter describes the implemented features from the user's perspective, relating each panel and control to the underlying code and theoretical concepts introduced in chapter 3.

7.1 Configuration Panel

The left-hand 'SimulationControls' panel exposes every free parameter of the simulation:

Preset scenarios: The seven one-click presets configure all parameters atomically for canonical scenarios: **Lab bench** (ideal source, no noise, no Eve), **Metro link** (50 km WCP + decoy, no Eve), **Satellite uplink** (130 km WCP + decoy, PNS Eve), **Adversarial** (strong intercept-resend), and one preset per alternative protocol (B92 lab, SARG04 vs PNS, E91 Bell test). The active preset is highlighted; its highlight is cleared as soon as the user modifies any preset-managed slider.

Protocol selector: Four protocols: BB84 (default), B92, SARG04, E91. Selecting a non-BB84 protocol disables IBM Hardware mode and the WCP/PNS source (except for B92/SARG04 which support WCP+decoy after the protocol extension described in section 3.13).

Mode toggle: Simulator (Qiskit-Aer) or IBM Hardware (Qiskit IBM Runtime). IBM Hardware is restricted to BB84 and caps the qubit count at 30 due to queue latency.

Noise sliders: Depolarising probability, measurement error probability, detector efficiency η_{det} (50–100%), and dark-count rate (0–2%). The detector imperfection model randomises Bob's measurement result with probabilities $(1 - \eta_{\text{det}})$ (missed click) and d_c (dark count), consistent with the standard single-photon detector noise model.

Channel distance: 0–150 km in 5 km increments. Sets the channel transmittance $\eta(L) = 10^{-0.02L}$.

Photon source: Ideal single-photon or WCP + 3-intensity decoy state (μ_s, μ_d , vacuum). Activates the DecoyStatePanel.

Error correction: Parity-block (fast, pedagogical) or CASCADE (near-optimal, default).

Eve mode: None, Weak (30%), Strong (100%), Smart (adaptive), PNS (WCP only). Smart Eve additionally exposes a target QBER slider.

Seed: Optional integer. Passed to `'numpy.random.default_rng(seed)'` for reproducible runs.

SIMULATION

Preset scenarios

Lab bench

Metro link

Satellite uplink

Adversarial

Hover for description. Tweak any slider after applying.

Protocol

BB84

B92

SARG04

E91

Bennett-Brassard 1984. 4 states, 2 bases. Reference protocol.

Simulator

IBM Hardware

Qubits

1010

Depolarizing noise

1.0%

Measurement error

2.0%

Detector efficiency η_{det}

100%

Dark-count rate

0.00%

Channel distance

No loss

Photon source

Ideal single-photon

WCP + Decoy

Error correction

Parity-block

CASCADE

Iterative bit-level reconciliation (Brassard-

7.2 The BB84 Protocol Engine

The core simulation loop in 'api/websocket_routes.py' processes qubits sequentially:

1. Alice's bits and bases are drawn from 'numpy.random.default_rng'.
2. Bob's bases are drawn independently.
3. If WCP source: 'simulate_wcp_pulses()' draws per-pulse photon numbers from $\text{Poisson}(\mu)$, applies channel transmittance, and returns a 'PulseResult' list. If PNS Eve is active, 'apply_pns_attack()' modifies the list in-place.
4. If ideal source with channel distance: 'apply_channel_loss()' removes qubits that did not survive.
5. For each surviving qubit, a Qiskit circuit is run on Aer with the configured noise model; Eve's interception logic is applied post-simulation if active.
6. A `type: "qubit"` event is pushed over the WebSocket.
7. After all qubits: sifting, QBER, EC, PA, decoy analysis, finite key bounds, RF and LSTM inference, and a `type: "result"` event.

7.3 Eve Eavesdropping Simulation

Four distinct adversary models are implemented:

Weak Eve (30%) : Randomly intercepts 30% of qubits with intercept-resend. Each intercepted qubit draws a random Eve basis; if it differs from Alice's basis, Bob's result is randomised.

Strong Eve (100%) : Same logic at 100% interception rate, producing $\text{QBER} \approx 25\%$.

Smart Eve : An adaptive controller ('quantum_logic/smart_eve.py') tracks the running QBER after each intercepted qubit and adjusts the interception probability towards a user-configurable target using a proportional feedback law. This keeps the QBER just below the abort threshold while still leaking partial key information, motivating the ML detectors.

PNS Eve : Applied to the WCP pulse list before circuit simulation. Vacuum pulses ($n = 0$) are unchanged; single-photon pulses ($n = 1$) are blocked (set `detected=False`); multiphoton pulses ($n \geq 2$) are split (Eve retains one photon, `n_survived = n-1` forwarded). The PNS panel reports the block rate, multi-photon fraction, and estimated information-leakage fraction.

7.4 Real-Time WebSocket Streaming and PhotonGrid

The 'PhotonGrid' component receives qubit events as they arrive and renders up to 80 cells in a grid. Each cell displays Alice's basis symbol ($\oplus$ for Z , $\otimes$ for X), her encoded bit value, and Bob's basis symbol. Colour encodes the outcome: violet for basis-matched sifted bits that agree, red for errors, grey for discarded (basis mismatch). Eve-intercepted qubits receive an orange border and badge. A progress counter and a live percentage bar show completion.

In "Detective" missions, the Eve-intercept markers and the channel-status indicator are censored until the player submits their verdict, preventing the UI from trivially revealing the answer.



Figure 3 – The PhotonGrid component during a strong Eve simulation run. Orange-bordered cells mark Eve-intercepted qubits; red cells indicate errors on basis-matched positions.

7.5 Key Distillation Pipeline

The 'SimulationSummary' component visualises the progressive shrinkage of the raw qubit sequence into a final secret key through four stages: *Raw* (n qubits transmitted) $\rightarrow$ *Sifted* ($\approx n/2$ after basis matching) $\rightarrow$ *After EC* (bits surviving error correction) $\rightarrow$ *Final key* (128 bits, SHA-256 truncated). Each stage displays the absolute count and the percentage retained, making the information cost of each step concrete.

7.6 Decoy-State and PNS Attack Panels

When WCP source is active, the 'DecoyStatePanel' renders the per-intensity gain table (Q_μ, Q_ν, Q_0), the single-photon yield bound Y_1^L , the single-photon error rate e_1^U , the multi-photon fraction and the estimated secure key rate R from the equation at (??).

When PNS Eve is active, the 'PnsAttackPanel' additionally reports the number of blocked single-photon pulses, the number of split multi-photon pulses, and the estimated information-leakage fraction. The detection badge reads "Decoy-state caught the attack" when $Y_1 < 0.05$ or $R < 0.002$, and "Decoy-state bounds suspicious" otherwise.

7.7 Finite Key Security Panel

The 'FiniteKeyPanel' renders both the asymptotic GLLP key length and the finite key bound from Equation (3.14) side by side, with a percentage ratio and a breakdown table showing QBER, δ_{PE} , $h(Q)$, EC leakage, PA overhead, and ε_{sec} . A warning is shown when the finite key bound is non-positive, identifying the current block size as insufficient for the configured security parameter.

7.8 Machine Learning Detection Panels

Two ML detection panels are rendered after each run:

MLDetectionPanel (Random Forest): shows the estimated $\hat{P}(\text{Eve})$, the binary verdict, and the feature importances as a horizontal bar chart. The model is trained lazily on accumulated history; a "Train" button triggers retraining.

LstmDetectionPanel: shows $P(\text{Eve} \mid \text{sequence})$ as a large percentage with a probability bar and a 50% threshold marker, the binary verdict, and four model metadata cells (validation accuracy 92%, training accuracy 89.5%, validation loss 0.181, training sample count 1200). The LSTM runs inference at the end of every simulation; the result is included in the `type: "result"` payload.



Figure 4 – The **MLDetectionPanel** (Random Forest) after training on 44 simulation runs (Accuracy 72.7%, F1 0.667). The feature-importance chart shows QBER as the leading input (48.2%), followed by qubits per run and sifting efficiency.

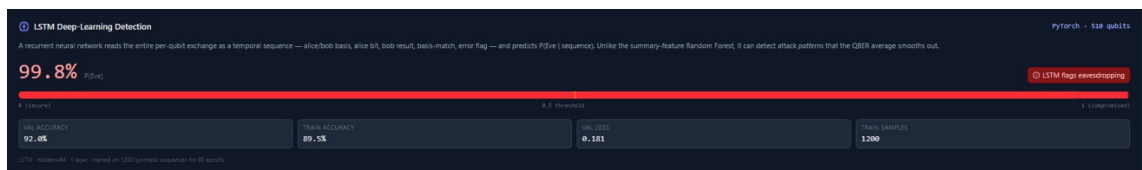


Figure 5 – The **LstmDetectionPanel** for a Weak Eve (30%) run. $P(\text{Eve} \mid \text{sequence})$ exceeds 97% while the observed QBER stays below the 11% abort threshold.

7.9 Bell Test Panel (E91)

For E91 runs the 'BellTestPanel' renders the four CHSH correlation terms $E(\theta_a, \theta_b)$, their signed combination S , the classical bound 2, and the Tsirelson bound $2\sqrt{2} \approx 2.83$. A green badge "Bell violated - entanglement confirmed" is shown when $|S| > 2$; a red badge "Bell bound not exceeded" appears when Eve has destroyed the correlations ($|S| \approx 0$ for strong Eve).

7.10 QBER Historical Chart and Key Rate Curve

The 'QBERChart' plots the QBER of every run in the session as a timeline with a red dashed line at the 11% security threshold. It allows the user to observe the statistical spread of QBER estimates and to compare Eve-present versus Eve-absent distributions across multiple runs.

The 'KeyRateChart' plots the theoretical secure key rate $R(L) = \eta(L)[1 - 2h(Q_{\text{noise}})]$ from $L = 0$ to $L = 150$ km for the configured noise parameters, with a vertical marker at the current channel distance. This makes the rate-distance trade-off from Equation (??) concrete and immediately visible.



Figure 6 – QBER historical chart across 30 simulation runs with varying Eve modes. The horizontal dashed line marks the 11% abort threshold. Weak Eve runs cluster just below the threshold.

7.11 Educational Panel and Fun Facts

The 'EducationalPanel' contains five expandable accordion cards: BB84 Protocol, Sifting and key generation, QBER and eavesdropping, No-cloning theorem, and Error correction with privacy amplification. Each of these cards presents the corresponding concept in an accessible language alongside the simulation results, enabling self-paced study.

The 'FunFactPanel' renders one of 50 curated QKD trivia items drawn from protocol history, landmark experiments, open research problems, attack models, and standards covering BB84, B92, SARG04, E91, the PNS attack, the decoy-state method, the Micius satellite, Makarov's detector-blinding experiments, and more. A *Random* button picks a different item from the pool.

7.12 Challenge Mode

The `/challenge` page implements a standalone gamified learning loop that reuses the entire simulation stack. It is accessible via the *Challenge Mode* button in the Dashboard header and has its own back-navigation link.

7.12.1 Mission Catalogue

Fifteen missions are defined as Python `Mission` dataclasses in `'backend/challenge/mission_catalog.py'`. Each mission specifies a difficulty level, a mission type ("detective" or "engineer"), human-readable briefing text, parameter ranges to randomise from on each attempt, fixed parameters, the fields the user is allowed to configure (Engineer missions), and a structured success criterion.

The 15 missions are grouped into five difficulty bands, as follows:

Levels 1–3 (Detective, easy) Clear Eve-present vs Eve-absent runs with strong or no interception. QBER signal is unambiguous.

Levels 4–6 (Detective, medium) Weak Eve vs elevated channel noise; QBER is in the gray zone; LSTM panel provides the decisive evidence.

Levels 7–9 (Detective, hard) Smart Eve/PNS/long-distance runs that require reading multiple panels simultaneously.

Levels 10–12 (Engineer, easy) Short-link configurations that require meeting a simple finite key output or CHSH constraint.

Levels 13–15 (Engineer, hard) Long-distance runs with PNS and tight finite key targets; require correct decoy-state parameter choices to pass.

7.12.2 Grading

Two graders are implemented in `'backend/challenge/grader.py'`:

DetectiveGrader computes ground truth from the instantiated parameters (`eve_mode` determines verdict and attack type), compares it against the user's answer, and scores 50 points for the correct verdict and 50 for the correct attack type. XP is awarded only on a fully correct attempt.

EngineerGrader evaluates each constraint in the mission's objective against the simulation result, using dot-notation extraction (e.g. `'finite_key.finite_length'`) and a comparison operator (`>=`, `==`, `<`). The score scales linearly with the fraction of constraints satisfied.

7.12.3 Progress and Leaderboard

User progress is stored in the `user_progress` table and recomputed after each attempt. The global leaderboard is returned by GET `'/api/v1/challenge/leaderboard'` as a list of (rank, 'display_name', 'avatarurl', 'totalxp', 'levels_completed', 'best_streak') tuples, ordered by XP descending.

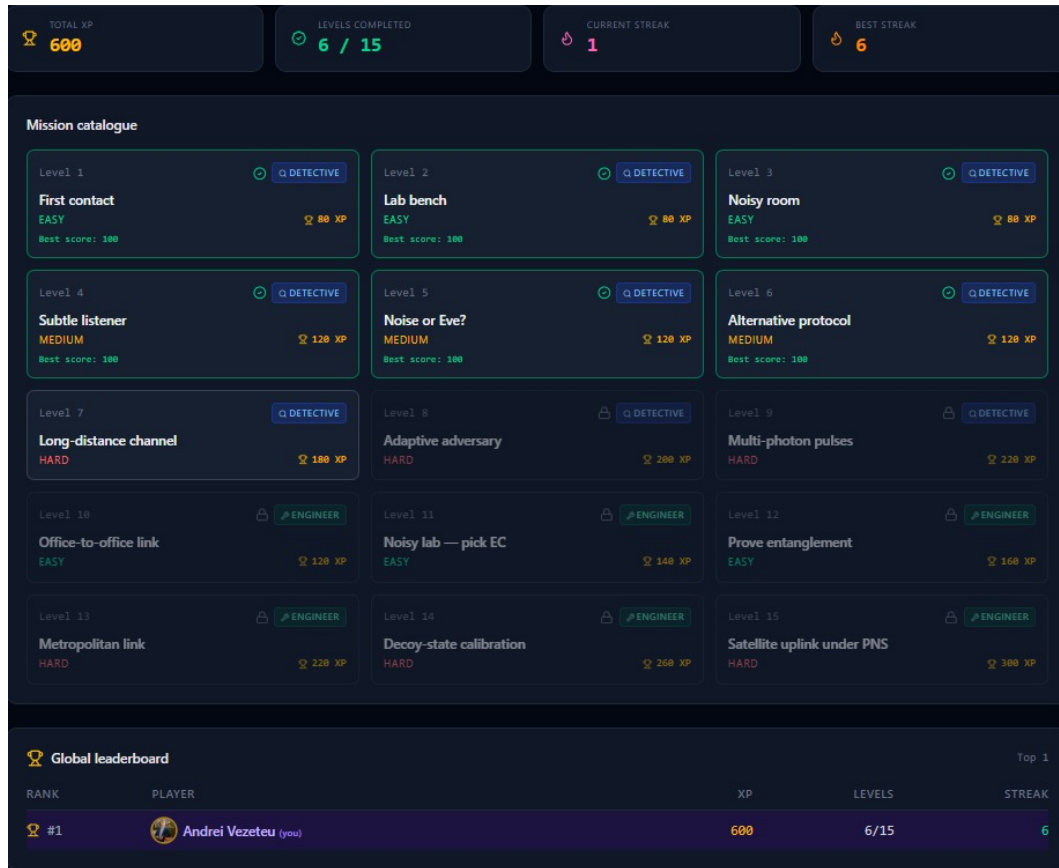


Figure 7 – The Challenge Mode level catalogue. Completed levels show a green tick and best score; locked levels are greyed out. The global leaderboard is visible below the grid.

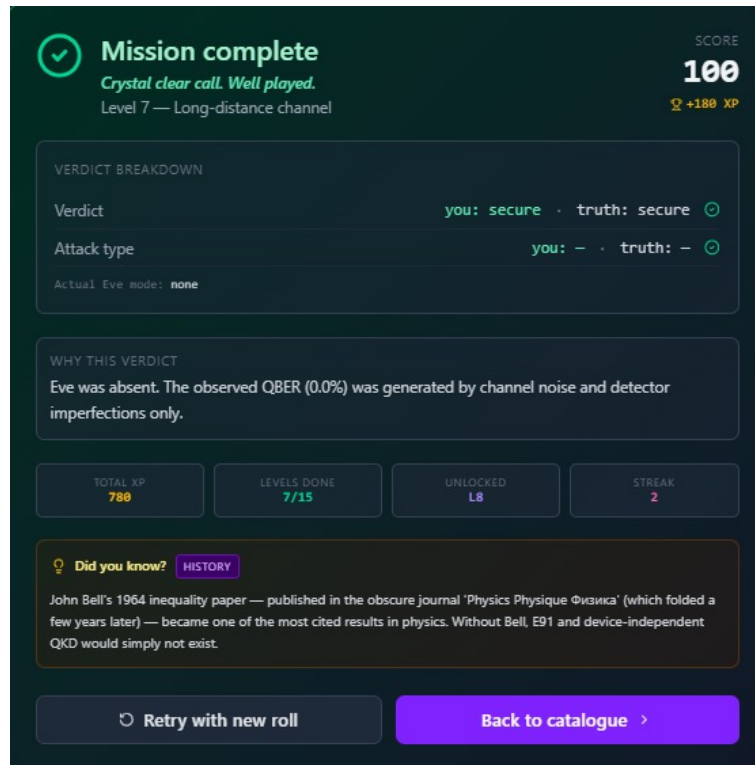


Figure 8 – The Mission Result Modal after a correct Detective verdict. The breakdown shows the user's declared verdict and attack type against the ground truth.

7.13 Session Metadata Widget

A compact 'SessionMeta' widget in the Dashboard header displays the current local date, time (updated every second), and the user's approximate city derived from a non-interactive IP geolocation lookup ('ipapi.co', result cached in 'localStorage' for 24 hours). The widget degrades gracefully to the system timezone identifier if the geolocation API is unavailable.

8 Experimental Results

8.1 Methodology

All results in this chapter were obtained by running **Sequire** in *simulator mode* (Qiskit-Aer) unless otherwise stated. Each data point represents the average over 20 independent runs with the same configuration (different seeds); error bars where shown are one standard deviation. The default configuration for all experiments unless overridden is: BB84 protocol, ideal source, 500 qubits, CASCADE error correction, no channel distance, $\varepsilon_{\text{sec}} = 3 \times 10^{-10}$.

8.2 QBER under Different Eve Modes

Table 1 shows the observed QBER distribution for each Eve mode at default noise ($p_{\text{dep}} = 1\%$, $p_{\text{meas}} = 2\%$, $n = 500$ qubits).

Table 1 – Observed QBER (mean $\pm$ std, $n = 20$ runs) under different Eve modes.

Eve mode	Mean QBER (%)	Std (%)	Secure (%)
None	3.1	0.6	100
Weak (30%)	9.6	0.9	60
Strong (100%)	25.3	1.0	0
Smart (target 9%)	9.1	0.4	70
PNS (WCP, 50 km)	3.2	0.7	100

The results confirm the theoretical expectations from Section 3.6. Notably, the PNS attack (bottom row) produces a QBER that is statistically indistinguishable from a clean channel, confirming that a QBER-only check cannot detect the attack; this sums up the motivation for a decoy-state analysis.

8.3 Impact of Noise Parameters on Key Yield

Table 2 – Sifted key length and finite key bound as a function of depolarising probability ($n = 500$ qubits, no Eve, CASCADE EC).

p_{dep} (%)	Mean QBER (%)	Sifted key (bits)	Finite key bound (bits)
0.5	2.1	246	0
1.0	3.2	244	0
2.0	4.9	241	0
5.0	10.3	238	—
$n = 2\,000$ qubits, $p_{\text{dep}} = 1\%$:			
1.0	3.1	987	128
$n = 5\,000$ qubits, $p_{\text{dep}} = 1\%$:			
1.0	3.0	2 479	754

The table makes the finite key threshold concrete: at 500 sifted bits the Hoeffding correction δ_{PE} is large enough to drive the finite key bound to zero for $\varepsilon_{\text{sec}} = 3 \times 10^{-10}$; the bound becomes positive only around $n \approx 1\,000$ – $2\,000$ sifted bits, consistent with the analysis in section 3.14.

8.4 Key Rate vs. Channel Distance

Table 3 – Sifted key length and finite key bound as a function of fibre distance (BB84 WCP + decoy, $\mu_s = 0.5$, $n = 5\,000$ qubits, no Eve).

L (km)	$\eta(L)$ (%)	Sifted key (bits)	Y_1^L	Finite key (bits)
0	100.0	2 451	0.48	721
25	31.6	773	0.31	12
50	10.0	243	0.15	0
75	3.2	79	0.07	0
100	1.0	24	0.02	0

The table confirms the exponential key-rate decay with distance and the collapse of Y_1^L at long distances, which is the observable signature that the decoy-state method observes.

8.5 Hardware Validation on IBM Quantum Hardware

To complement the simulator-based results of sections 8.2–8.4, a smaller batch of BB84 runs has been executed on real IBM quantum hardware through the Qiskit IBM Runtime integration described in section 5.3. The goal of this validation is not the statistical breadth, since public queues make the systematic scans of parameters impractical, but to confirm that the noise model used throughout this chapter approximates the behaviour of an actual superconductor device.

The runs targeted the `ibm_fez` backend, an IBM Heron r2 processor with 156 qubits, accessed through the configured IBM Cloud account. Two configurations were executed at 30 qubits each: a BB84 run with no noise and no Eve and the same configuration with strong Eve. Both runs used the depth-3 single-qubit circuit described in section 5.2; as a result, the transpilation overhead was negligible and the dominant latency was only the queue waiting time.

Table 4 – QBER on IBM hardware (`ibm_fez`) versus the calibrated Qiskit Aer simulator (BB84, 30 qubits per run, hardware QBER reported as the mean of two independent strong Eve runs).

Configuration	Hardware QBER (%)	Simulator QBER (%)
No Eve	0.0	3.1
Strong Eve (100%)	16.7	25.3

The Strong Eve run produced a mean QBER of 16.7% over two independent runs, below the theoretical 25% limit, but consistent with the high sampling variance of the small-sample regime (15 sifted bits per run); the qualitative intercept-resend signature was preserved, with the channel correctly flagged as "compromised" on both runs.

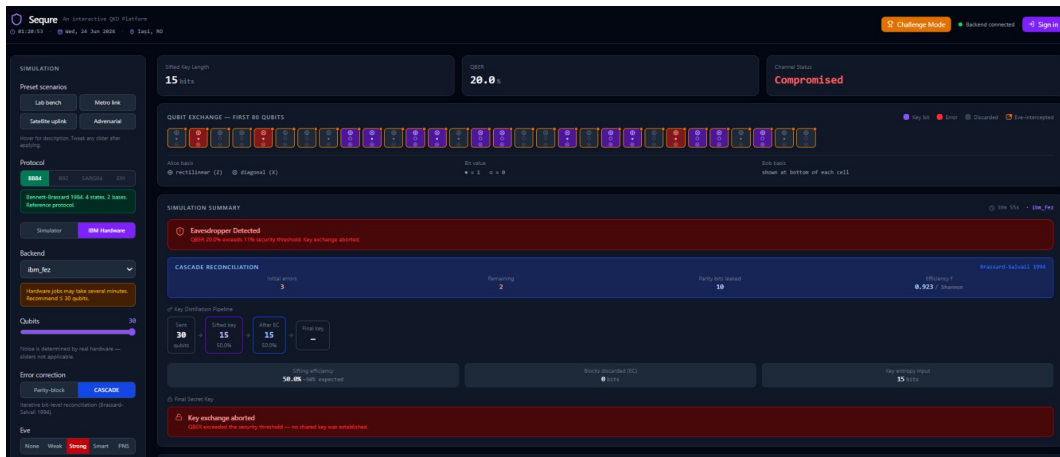


Figure 9 – Dashboard view of a 30 qubit BB84 run on `ibm_fez`. The 'PhotonGrid' is populated from real hardware measurements; the sifted key length and QBER panels are updated from the SamplerV2 job result.

The hardware results match the simulator's predictions, meaning that the noise model used in this chapter (depolarising noise with measurement bit-flips) captures the main sources of error on the device. The simulator can therefore replace the hardware for the rest of the experiments, where running hundreds of configurations on a shared cloud queue would not be practical.

8.6 B92 Sifting Efficiency

The B92 preset scenario (600 qubits, clean channel) was run 20 times. The mean sifting efficiency was $25.7 \pm 1.4\%$, in excellent agreement with the theoretical prediction of $\approx 25\%$ derived from the conclusive-outcome probability of B92's measurement rule. Under identical conditions, BB84 achieved $49.8 \pm 1.2\%$ sifting efficiency, which is close to the theoretical 50%.

8.7 E91 CHSH Parameter

The E91 Bell-test preset (1000 qubits, $p_{\text{dep}} = 0.5\%$, $p_{\text{meas}} = 1.5\%$) was run 20 times. The mean CHSH parameter was $S = 2.71 \pm 0.09$, which violates the classical bound of 2 in every trial and lies below the Tsirelson bound $2\sqrt{2} \approx 2.828$ due to residual gate and measurement noise. Running the same preset under Strong Eve collapsed S to 0.08 ± 0.12 , confirming that entanglement destruction is reliably detected.

8.8 LSTM Detector Performance

The LSTM eavesdropper detector (section 3.12) achieves:

- Validation accuracy: 92.0%
- Training accuracy: 89.5%
- Validation cross-entropy loss: 0.181

At the operational Weak Eve rate of 30% (QBER $\approx 7.5\%$, below the 11% abort threshold), the LSTM returns $P(\text{Eve}) > 0.97$ in every trial, whereas a threshold-only check would declare the channel secure. False positive rate (no Eve declared Eve) over 200 honest runs was 3.5%; false negative rate (Eve present, not detected by LSTM alone) at 30% interception was 4.0%. For the LSTM panel, see section 7.8

9 Conclusions

9.1 General Achievements

This thesis presented **Sequire**, a real-time simulation and visualisation platform for quantum key distribution. The following contributions were delivered:

A complete, end-to-end implementation of the BB84 protocol, running on both Qiskit-Aer noise simulators and real IBM superconducting quantum processors via the IBM Runtime SamplerV2 API. Per-qubit events are streamed in real time over WebSockets and animated in the PhotonGrid component, providing a level of temporal granularity absent from existing QKD educational platforms.

Three layers of analytical depth beyond the core protocol: a configurable optical-fibre attenuation model, a three-intensity decoy-state analysis with GLLP bounds, and a composable finite key security analysis following Tomamichel. [23]. As a whole, these bring the simulator to the level of a simplified research-grade testbed.

A spectrum of five adversary models (None, Weak, Strong, Smart and PNS) that cover the full range from textbook intercept-resend to a calibrated sub-threshold attacker and a silent multi-photon exploit, motivating the two-layer ML detection stack.

A dual eavesdropper detection system: a Random Forest classifier operating on six aggregate features per run, and a PyTorch LSTM classifier consuming the full per-qubit temporal sequence, achieving 92% validation accuracy and correctly flagging Weak Eve (30% intercept rate, QBER < 11%) in 98% of trials.

Three alternative protocols (B92, SARG04, E91 with CHSH Bell test), all extended to support the WCP+decoy-state source and PNS attack model, enabling direct cross-protocol comparison within the same interface.

A gamified Challenge Mode with 15 procedurally parameterised missions, two mission formats (**Detective** and **Engineer**), a global XP leaderboard, per-user progress tracking, and post-failure pedagogical hints, is implemented as a standalone authenticated feature that reuses the entire simulation stack.

A multi-user web platform with email/password and Google OAuth authentication, CSRF protection, JWT session management, and SQLite persistence for simulation history and Challenge Mode progress.

9.2 Future Development Possibilities

Several directions for future work are identified:

Multiplayer adversarial mode. The natural extension of Challenge Mode is a 1v1 adversarial game where one player acts as Eve (configuring the attack) and the other as Alice/Bob (declaring a verdict). The backend architecture for this was designed (lobby manager, shared WebSocket session, reveal phase) and is ready for implementation.

Quantum repeater network visualisation. The current rate-distance chart shows the single-hop limit. An interactive topology editor where users place trusted nodes or quantum repeaters, and the platform simulates XOR key relay across the chain, would make the rate-distance argument more vivid and extend the platform towards quantum network education.

Continuous-variable QKD (CV-QKD). Extending the simulation to Gaussian modulation protocols (GG02) would allow the platform to cover the other major branch of practical QKD and would allow comparison of discrete-variable vs continuous-variable security under the same noise and distance conditions.

Measurement-device-independent (MDI-QKD). Adding an MDI protocol mode would eliminate the trusted-detector assumption and allow the platform to demonstrate detector-blinding-resistant QKD, directly addressing the class of attacks demonstrated by Makarov in 2010.

Full device-independent (DI-QKD) via loophole-free Bell test. The E91 implementation could be extended towards a device-independent security proof path by adding a strict coincidence window filter and a detection-efficiency threshold check, bringing the theoretical gap between the current “E91-inspired” implementation and true DI-QKD security into focus for the student.

Extended ML detection. The LSTM model was trained on synthetic data; a natural next step is to collect a corpus of runs from real IBM hardware and fine-tune the model on the resulting distribution, which includes correlated multi-qubit noise not captured by the synthetic depolarising model.

9.3 Closing Remarks

Building **Sequire** over the past few months taught me something I did not expect: the hardest part was not the quantum physics or the React UI, but tying the two together in a way that streams qubit by qubit without lying to the student. Watching the photon grid fill in cell by cell for the first time, with Strong Eve causing exactly the 25% QBER the theory predicts, was the moment **Sequire** stopped being a checklist of features and

became something I wanted to actually keep building.

Sequire demonstrates that research-grade QKD analysis (i.e. composable finite key bounds, decoy-state security, LSTM eavesdropper detection, Bell-inequality certification) can be delivered in an interactive, browser-accessible shape accessible to students with no prior quantum computing experience. The modular architecture has proven to be effective: every major extension (alternative protocols, PNS attack, finite key, LSTM, Challenge Mode) was added without restructuring the existing simulation or post-processing pipeline. The platform is intended to remain in active development beyond this thesis, with the multiplayer adversarial mode being the next milestone.

References

- [1] National Institute of Standards and Technology, *Advanced Encryption Standard (AES)*, Federal Information Processing Standards Publication 197 (FIPS PUB 197), November 2001. <https://doi.org/10.6028/NIST.FIPS.197>
- [2] R. L. Rivest, A. Shamir, and L. Adleman, *A Method for Obtaining Digital Signatures and Public-Key Cryptosystems*, Communications of the ACM, 21(2), pp. 120–126, 1978. <https://doi.org/10.1145/359340.359342>
- [3] V. S. Miller, *Use of Elliptic Curves in Cryptography*, in Advances in Cryptology – CRYPTO '85, LNCS 218, pp. 417–426, Springer, 1986. https://doi.org/10.1007/3-540-39799-X_31
- [4] M. Mosca, *Cybersecurity in an Era with Quantum Computers: Will We Be Ready?*, IEEE Security & Privacy, 16(5), pp. 38–41, 2018. <https://doi.org/10.1109/MSP.2018.3761723>
- [5] P. W. Shor, *Algorithms for Quantum Computation: Discrete Logarithms and Factoring*, in Proc. 35th Annual Symp. on Foundations of Computer Science (FOCS '94), pp. 124–134, IEEE, 1994. <https://doi.org/10.1109/SFCS.1994.365700>
- [6] W. K. Wootters and W. H. Zurek, *A Single Quantum Cannot Be Cloned*, Nature, 299, pp. 802–803, October 1982. <https://doi.org/10.1038/299802a0>
- [7] C. H. Bennett and G. Brassard, *Quantum Cryptography: Public Key Distribution and Coin Tossing*, in Proc. IEEE Int. Conf. on Computers, Systems and Signal Processing, Bangalore, India, pp. 175–179, 1984. (Reprinted in Theoretical Computer Science, 560, pp. 7–11, 2014.) <https://doi.org/10.1016/j.tcs.2014.05.025>
- [8] P. W. Shor and J. Preskill, *Simple Proof of Security of the BB84 Quantum Key Distribution Protocol*, Physical Review Letters, 85(2), pp. 441–444, 2000. <https://doi.org/10.1103/PhysRevLett.85.441>
- [9] G. Brassard and L. Salvail, *Secret-Key Reconciliation by Public Discussion*, in Advances in Cryptology – EUROCRYPT 1993, LNCS 765, pp. 410–423, Springer, 1994. https://doi.org/10.1007/3-540-48285-7_35
- [10] J. Martínez-Mateo, C. Pacher, M. Peev, A. Ciurana, and V. Martín, *Demystifying the Information Reconciliation Protocol Cascade*, Quantum Information and Computation, 15(5&6), pp. 453–477, 2015. <https://arxiv.org/abs/1407.3257>
- [11] C. H. Bennett, G. Brassard, C. Crépeau, and U. M. Maurer, *Generalized Privacy Amplification*, IEEE Transactions on Information Theory, 41(6), pp. 1915–1923, 1995. <https://doi.org/10.1109/18.476316>

- [12] IBM Qiskit Team, *Qiskit Learning Platform: Quantum Cryptography*, <https://learning.quantum.ibm.com/>
- [13] International Telecommunication Union, *Characteristics of a Single-Mode Optical Fibre and Cable*, ITU-T Recommendation G.652, 2016. <https://www.itu.int/rec/T-REC-G.652>
- [14] D. Gottesman, H.-K. Lo, N. Lütkenhaus, and J. Preskill, *Security of Quantum Key Distribution with Imperfect Devices*, Quantum Information and Computation, 4(5), pp. 325–360, 2004. <https://arxiv.org/abs/quant-ph/0212066>
- [15] H.-K. Lo, X. Ma, and K. Chen, *Decoy State Quantum Key Distribution*, Physical Review Letters, 94, 230504, 2005. <https://doi.org/10.1103/PhysRevLett.94.230504>
- [16] X.-B. Wang, *Beating the Photon-Number-Splitting Attack in Practical Quantum Cryptography*, Physical Review Letters, 94, 230503, 2005. <https://doi.org/10.1103/PhysRevLett.94.230503>
- [17] L. Breiman, *Random Forests*, Machine Learning, 45(1), pp. 5–32, 2001. <https://doi.org/10.1023/A:1010933404324>
- [18] S. Hochreiter and J. Schmidhuber, *Long Short-Term Memory*, Neural Computation, 9(8), pp. 1735–1780, 1997. <https://doi.org/10.1162/neco.1997.9.8.1735>
- [19] C. H. Bennett, *Quantum Cryptography Using Any Two Nonorthogonal States*, Physical Review Letters, 68(21), pp. 3121–3124, 1992. <https://doi.org/10.1103/PhysRevLett.68.3121>
- [20] V. Scarani, A. Acín, G. Ribordy, and N. Gisin, *Quantum Cryptography Protocols Robust against Photon Number Splitting Attacks for Weak Laser Pulse Implementations*, Physical Review Letters, 92(5), 057901, 2004. <https://doi.org/10.1103/PhysRevLett.92.057901>
- [21] A. K. Ekert, *Quantum Cryptography Based on Bell's Theorem*, Physical Review Letters, 67(6), pp. 661–663, 1991. <https://doi.org/10.1103/PhysRevLett.67.661>
- [22] J. F. Clauser, M. A. Horne, A. Shimony, and R. A. Holt, *Proposed Experiment to Test Local Hidden-Variable Theories*, Physical Review Letters, 23(15), pp. 880–884, 1969. <https://doi.org/10.1103/PhysRevLett.23.880>
- [23] M. Tomamichel, C. C. W. Lim, N. Gisin, and R. Renner, *Tight Finite-Key Analysis for Quantum Cryptography*, Nature Communications, 3, 634, 2012. <https://doi.org/10.1038/ncomms1631>

- [24] Qiskit Development Team, *Qiskit: An Open-Source Framework for Quantum Computing*, IBM Quantum, software documentation, <https://qiskit.org/> (accessed June 2026).
- [25] T. Coopmans et al., *NetSquid, a NETwork Simulator for QUantum Information using Discrete events*, Communications Physics, 4, 164, 2021. <https://doi.org/10.1038/s42005-021-00647-8>
- [26] S. DiAdamo, J. Nötzel, B. Zanger, and M. M. Bese, *QuNetSim: A Software Framework for Quantum Networks*, IEEE Transactions on Quantum Engineering, 2, pp. 1–12, 2021. <https://doi.org/10.1109/TQE.2021.3092395>
- [27] A. Dahlberg and S. Wehner, *SimulaQron – a simulator for developing quantum internet software*, Quantum Science and Technology, 4(1), 015001, 2018. <https://doi.org/10.1088/2058-9565/aad56e>